

Article

On the Performance of YOLO and ML/DL Models for Lightweight, Real-Time Smoke and Fire Detection on an x86-Based Edge Platform: An Explainable Decision-Level Sensor Fusion Framework

Endri Dibra and Panagiotis K. Gkonis * 

Department of Digital Industry Technologies, National and Kapodistrian University of Athens, Dirfies Messapies, 34400 Psachna, Greece; endri7dibra@gmail.com

* Correspondence: pgkonis@uoa.gr

Abstract

In this study, a decision-level detection framework is presented and evaluated; it integrates sensor data (e.g., temperature, humidity, gas readings) with machine learning (ML) models and computer vision-based smoke and fire detection systems, in an effort to increase overall robustness, as well as false-alarm reduction. To this end, sixteen (16) ML and deep learning (DL) models are employed on an Internet of Things (IoT) sensor dataset. Moreover, a range of YOLO (You Only Look Once) models, such as older versions (YOLOv5n, YOLOv8n), as well as newer versions (YOLOv10n, YOLOv11n, YOLOv12n), are employed on an image-label-based dataset. Model selection initially prioritizes lightweight architectures that are suitable for resource-constrained edge devices. Afterwards, the selected models are evaluated via well-known metrics, such as parameter count, F1-score/mean average precision (mAP) and real-time inference latency. In the same context, explainable AI (XAI) techniques, such as SHAP (SHapley Additive exPlanations) for ML models and LIME (Local Interpretable Model-agnostic Explanations) for the YOLO detectors, are integrated into the platform as well. According to the presented results, the Explainable Sensor Fusion (ESF) framework demonstrates a high level of internal consistency and logical reliability through its decision-level fusion paradigm in a controlled environment.

Keywords: YOLO; sensor fusion; machine learning (ML); explainable AI (XAI); edge devices



Academic Editors: Dong-Sung Pae, Sungju Lee and Stefano Scanzio

Received: 17 December 2025

Revised: 30 January 2026

Accepted: 5 February 2026

Published: 10 February 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

Industrial facilities handling volatile materials, complex machinery, and high-value assets are particularly vulnerable to fire accidents, which eventually may result in significant losses and worker casualties. In regions where manufacturing is a significant contributor to the local economy, the frequency of these events necessitates robust and reliable fire detection technologies, in order to prevent and handle these damages in proper ways.

In this context, the development of effective smoke and fire detection systems has evolved rapidly over the last decade, mainly due to technological advances in machine learning (ML), deep learning (DL) and computer vision techniques on Internet of Things (IoT) devices [1]. To this end, traditional sensor-based approaches relying on thresholds for temperature, humidity, or gas levels may offer reliable but isolated results, sometimes leading to false alarms. On the other hand, ML and DL models, such as random forest,

gradient boosting ensembles and neural networks (NNs), can provide improved accuracy due to their inherent ability to process multivariate sensor data, thus achieving high accuracy and fast responses depending on the computational capability of the processing nodes. However, it should be mentioned at this point that real-time deployments of such approaches remain an open issue, due to the vast amount of data that needs to be collected, stored and processed on IoT, edge and cloud devices. To this end, object detection frameworks like the YOLO (You Only Look Once) family [2–6] have revolutionized vision-based monitoring with nano variants (e.g., YOLOv5n, YOLOv8n, YOLOv10n, YOLOv11n, YOLOv12n) by significantly reducing deployment requirements in lightweight devices. To this end, YOLO approaches are capable of spotting smoke wisps [7] or flame flickers in RGB feeds at 20+ frames per second.

Despite these advancements, the development of a highly accurate and computationally efficient fire detection scheme may face several critical challenges. For example, the latest versions of YOLO models (e.g., YOLOv11n, v12n) might not be always feasible to be integrated in resource-constrained devices. In the same context, although newer models often result in higher Mean Average Precision (mAP) compared to previous versions, they frequently require more complex optimization techniques (like TensorRT quantization) [8] that may have a direct impact on deployment complexity. This challenge is a central theme in recent industrial AI research, where the focus has shifted toward “device-aided” data collection and the design of specialized lightweight networks to ensure real-time viability on edge hardware [9]. Such studies emphasize that for industrial monitoring, ranging from infrastructure defect detection to emergency response, the system’s effectiveness is defined by its ability to maintain high inference throughput in specialized monitoring devices without relying on high-latency cloud processing. Furthermore, the decision-making processes of advanced ML/DL models and object detectors often remain a “black-box” issue.

Based on the above, this study introduces a decision-level (late) Explainable Sensor Fusion (ESF) framework for industrial emergency response cases on resource-constrained hardware devices. To this end, the main contributions of our work are listed below:

1. Performance evaluation of sixteen (16) ML/DL classifiers and a range of YOLO nano variants (v5n, v8n, v10n, v11n, v12n) is taking place, by prioritizing lightweight architectures and real-time inference viability on edge hardware [10–13].
2. We propose a robust fusion strategy, which integrates heterogeneous sensor outputs with conditional override logic, combining a weighted product rule for high-precision multimodal confirmation.
3. We apply explainable AI (XAI) techniques, specifically SHAP (SHapley Additive ex-Planations) [14,15] for tabular sensor decisions and LIME (Local Interpretable Model-agnostic Explanations) [15,16] to provide interpretable and operator-friendly outputs.

The rest of this work is organized as follows: In Section 2, the theoretical background is provided for all the components that constitute the deployed alarm detection scheme. In Section 3, the methodology for performance evaluation is discussed along with specific hardware requirements. Results are presented in Section 4 for a variety of ML models and YOLO versions. Discussion takes place in Section 5, while concluding remarks along with proposals for future work are discussed in Section 6.

2. Theoretical Background

In this section, the related theoretical background is provided to develop a lightweight, real-time smoke and fire detection system for edge devices. To this end, open-source AI frameworks, state-of-the-art ML, DL, as well as detection models, and advanced fusion and explainability techniques are presented.

2.1. Open-Source AI Frameworks

DL implementations are frequently based on open access frameworks like TensorFlow (v. 2.15) [17], an end-to-end platform which is optimized for high-performance numerical computation tasks. It utilizes an efficient C++ backend system to execute operations defined via a Python interface (v. 3.11.9), enabling the flexible construction of dataflow graphs. Moreover, TensorFlow's environment is easy to use, offering tools like TensorFlow Lite/Serving (v. 2.15) for streamlined model deployment across mobile and edge environments. Another framework is Keras (v. 2.15) [18], a high-level, user-friendly API designed for rapid experimentation and development of DL models such as convolutional neural networks (CNNs) or recursive neural networks (RNNs). Finally, Scikit-Learn (v. 1.3.2) [19] is a well-known Python library that provides a unified interface to a comprehensive set of algorithms for classification, regression, and pre-processing tasks.

2.2. ML/DL Approaches for Environmental Sensing

For our sensor-based binary classification, a wide range of models was evaluated to identify the most efficient lightweight architecture approach for edge inference. These models are listed below:

- Linear models like Logistic Regression [20] and RidgeClassifier [21] are two well-known models for sensor data processing. In particular, the RidgeClassifier is a robust regularized linear classifier that minimizes loss augmented by an L2 penalty term, which shrinks feature weights to mitigate overfitting. Then, the non-linear approaches like KNeighborsClassifier [22] capture non-linear decision boundaries based on Euclidean distance, while the Support Vector Classifier (SVC) [23] uses the kernel trick to map raw sensor features into a higher-dimensional space for linear separability.
- Tree-based methods, such as the DecisionTreeClassifier [24], that may be employed directly on unscaled sensor data, can provide an interpretable, white-box model. Additionally, the RandomForestClassifier [25] is an advanced ensemble method that operates by constructing a multitude of independent Decision Trees, each trained on a random subset of data (bagging), reducing variance and mitigating the weakness of single, overfit trees.
- Boosting techniques are sequential ensemble methods designed to significantly enhance performance through iterative error correction. More precisely, XGBoost (eXtreme Gradient Boosting) [26] is a scalable implementation that employs a regularized objective function to control model complexity (L1 and L2 regularization), making it robust against overfitting. After that, LightGBM (Light Gradient Boosting Machine) [27] improves speed by using Gradient-based One-Side Sampling (GOSS) and Exclusive Feature Bundling (EFB), making it highly efficient for handling massive datasets. Finally, CatBoost [28] uses Ordered Boosting with a permutation-driven approach to compute leaf values, effectively mitigating the problem of target leakage which is a key issue in Gradient Boosting Decision Trees (GBDTs).
- DL models, such as the MLPClassifier [29] and the TF-Keras Neural Network [30], implement Feedforward Neural Network (FNN) architectures that establish complex, non-linear decision boundaries through fully connected layers interconnected by non-linear activation functions.

2.3. Real-Time Object Detection Architectures (YOLO Nano)

The smoke and fire detection task is highly latency-critical, thus justifying the use and importance of the YOLO (You Only Look Once) architecture, which treats detection as a regression problem in a single network pass. Below, we present the YOLO nano models used in this work:

- YOLOv5nu: This version represents the most lightweight configuration among the evaluated models, emphasizing minimal parameter count and reduced computational complexity. Its architecture follows the traditional YOLO design, consisting of a compact convolutional backbone for feature extraction, a simplified feature aggregation neck, and a single-stage detection head. The backbone employs shallow depth and narrow channel widths, which significantly reduce FLOPs and memory footprint. While this design enables high throughput and real-time inference on resource-constrained devices, it limits the model's representational capacity, particularly for complex or small-scale targets. Consequently, YOLOv5nu often exhibits faster convergence during training but may show higher localization and classification loss compared to deeper variants. This trade-off explains its strong inference speed at the expense of reduced detection accuracy in challenging scenarios [31].
- YOLOv8n: This version introduces architectural refinements aimed at improving the balance between accuracy and efficiency. Compared to YOLOv5nu, it adopts an optimized backbone with improved feature reuse and a more effective feature fusion strategy in the neck. These changes enhance multi-scale feature representation while maintaining a lightweight structure suitable for edge deployment. The improved feature aggregation allows YOLOv8n to learn richer spatial and semantic information, leading to more stable training loss convergence and improved localization accuracy. As a result, YOLOv8n typically achieves higher mean average precision (mAP) than YOLOv5nu with only a modest increase in computational cost, making it a strong baseline for real-time vision tasks [32].
- YOLOv10n: This version introduces a fundamental architectural innovation through the consistent dual assignment strategy, which enables Non-Maximum Suppression (NMS)-free training and inference. Unlike traditional YOLO models that rely on post-processing to remove redundant detections, YOLOv10n integrates assignment consistency directly into the detection head design. This architectural choice reduces post-processing overhead and eliminates NMS as a performance bottleneck during inference. From a training perspective, the dual assignment mechanism improves label-matching consistency, which can lead to smoother loss curves and more stable optimization. In conclusion, the absence of NMS improves latency and scalability, particularly in real-time and high-frame-rate applications, while maintaining competitive detection accuracy [33,34].
- YOLOv11n: This version enhances feature extraction and representation by incorporating the C3K2 Block and the C2PSA (Cross Stage Partial with Spatial Attention) Block. These modules improve the model's ability to focus on spatially relevant regions by selectively emphasizing informative features while suppressing background noise. The integration of spatial attention mechanisms is particularly beneficial for detecting irregularly shaped and diffuse targets, such as smoke plumes and spreading fire. During training, these attention-enhanced blocks facilitate more precise gradient propagation to relevant regions, often resulting in improved localization loss and higher recall. Although these additions slightly increase computational complexity, the resulting gains in inference accuracy justify their inclusion for complex visual environments [35,36].
- YOLOv12n: Finally, this version further advances architectural efficiency by integrating a high-speed Area Attention (A^2) module and FlashAttention mechanisms. These components enable the model to capture global contextual relationships across the feature map while minimizing memory access overhead and computational latency. The A^2 module allows YOLOv12n to model long-range dependencies, which improves detection performance in scenes with spatially distributed or partially occluded objects.

FlashAttention further optimizes attention computation, ensuring that global context modeling does not significantly impact inference speed. As a result, YOLOv12n achieves improved accuracy and robustness while preserving real-time performance, making it particularly suitable for complex and dynamic vision tasks [37,38].

2.4. Multimodal Sensor Fusion and Evaluation

In real-world industrial environments, false alarms represent a critical challenge, as factories typically contain heavy machinery, airborne dust, steam, reflective metallic surfaces, and dynamic lighting conditions. These factors can significantly degrade the reliability of vision-based fire detection systems, where strong light reflections, hot surfaces, welding sparks, or motion blur may be misclassified as flames or smoke by RGB-based detectors such as YOLO. At the same time, environmental sensors may also produce spurious readings due to benign industrial processes, including ventilation flows, particulate emissions, or transient gas concentrations. As a result, relying on a single modality often leads to elevated false alarm rates. To address this, the proposed sensor fusion framework explicitly combines visual confidence with sensor-derived probabilistic evidence, enforcing confirmation across modalities before issuing an alert. This complementary design significantly reduces false positives caused by visual artifacts or sensor noise, while preserving sensitivity to genuine fire and smoke events, thereby improving operational reliability in complex industrial scenarios. More specifically, the framework uses a weighted-multiplicative fusion rule. The fused probability, P_{fusion} , is calculated as follows:

$$P_{fusion} = ((P_{sensor}^{0.55}) \times (P_{yolo}^{0.45})) \quad (1)$$

The slight priority given to sensor measurements reflects their better robustness in industrial environments where visual stability may be compromised. Finally, the system incorporates a high-confidence visual override mechanism for rapid response, compensating for situations where flames are visually obvious, but sensor data is delayed or out of the region.

Additionally, for evaluation purposes, the system utilizes various distinct metrics that include Accuracy, Precision, Recall, F1-Score, and ROC AUC (Receiver Operating Characteristic Area Under the Curve). For YOLO, the primary metrics are mAP, specifically mAP@0.50, mAP@0.50-0.95 and FPS (Frames-per-Second), which simultaneously evaluate classification, localization and real-time response and quality.

2.5. Explainable AI (XAI) and Hyperparameter Optimization

To develop a white-box system, we have incorporated explainability approaches, which are crucial for safety-critical systems. To this end, SHAP leverages game theory to assign a prediction to individual features by calculating the Shapley value. SHAP provides a unified, theoretically sound framework that computes the marginal contribution of each feature to the prediction [39]. On the other hand, LIME addresses the black-box problem by providing human-interpretable justifications. In particular, it operates on the premise that while a model may be non-linear globally, its behavior can be approximated by a simpler interpretable model (e.g., linear regression) within the immediate, local neighborhood of a specific instance. LIME achieves this in computer vision by generating perturbed images (e.g., masking super-pixel regions) to train the local explainer [40].

Finally, to utilize optimization methods on complex models, it is required to use advanced Hyperparameter Optimization (HPO). To achieve this feature, we employ Optuna, distinguished by its define-by-run API, which enables the dynamic construction of search spaces. More precisely, Optuna uses sophisticated sampling algorithms such as the Tree-structured Parzen Estimator (TPE) and a robust pruning mechanism to intelligently

and adaptively explore the hyperparameter space, reducing computational resources and accelerating convergence [41].

3. Materials and Methodology

In this section, the hardware specifications along with all related configurations are described towards the construction of the proposed lightweight real-time hybrid smoke and fire detection system for resource-constrained edge devices. All related code, trained models, and processed datasets are publicly available via this GitHub repository [42]. The analysis and development were conducted in Python 3.11+ environments, leveraging libraries and frameworks detailed in Section 2.1.

3.1. Hardware Specifications

To simulate and approach the resource constraints of an edge-computing environment, we performed all the real-time inference latency and performance benchmarks on a Dell Laptop with the following specifications:

- CPU: Intel Core (TM) i5-8365U
- Clock Speed: 1.60 GHz
- RAM: 16 GB
- Storage: 256 GB SSD

3.2. Datasets

In the context of this work, we made use of two different datasets, with one being utilized for the ML/DL models and the second one for the YOLO nano models. The ML/DL models use a .csv file containing the sensor's readings in integer and float-point number format. On the other hand, detection models, like the YOLO nano versions, need a dataset that combines each image with a label that provides the class name [class_id, e.g., 0: fire or 1: smoke] and the coordinates [x_center, y_center, width, height] of the bounding boxes, which belong to the detected smokes and fires on the image. As a result, a detection model learns what a smoke or fire looks like, but at the same time becomes able to capture and surround each of them uniquely with box limits. Finally, after training the models for both cases, they will be able to detect possible spikes of smoke and fire in the environment/air and on the camera/video frames, respectively.

3.2.1. Sensor Dataset

The dataset used to train and evaluate the ML and DL models [43], includes time-series from different sensors' readings from an environmental monitoring setup, simulating real-world conditions. In particular, it includes 62,629 instances across 13 features, such as the following: Temperature (°C), Humidity (%), TVOC (ppb), eCO₂ (ppm), Pressure (hPa), PM1.0 (µg/m³), PM2.5 (µg/m³), NC0.5 (#/cm³), NC1.0 (#/cm³), NC2.5 (#/cm³), Raw H₂ (raw ADC), and Raw Ethanol (raw ADC).

Additionally, the binary target label, "Fire Alarm," indicates fire/smoke events (1: alarm triggered; 0: normal), which are determined according to the features' values; thus, for instance, if there are high temperature (°C) and low humidity (%) levels, then it is determined that this particular case is prone to a fire event and vice versa. Moreover, the pre-processing part involved dropping irrelevant metadata (e.g., timestamps, counters) as well as the median imputation for <0.1% missing values.

The dataset was finally split into a (80/20 train/test) format using scikit-learn's 'train_test_split' with 'random_state = 42' and 'stratify = y' to maintain class balance (~5% positive events, reflecting real-world rarity). A correlation heatmap (Pearson coefficients) was generated via Seaborn/Matplotlib (v. 0.13.2/v.3.9.2) for exploratory analysis, revealing

moderate correlations (e.g., TVOC-eCO₂: $r = 0.45$) but no multicollinearity issues (Variance Inflation Factor (VIF) < 5).

3.2.2. Image Dataset for Object Detection

For our vision-based detection with YOLO nano models approach, we utilized the “New Fire Dataset” [44]. This dataset contains annotated RGB images, namely 5923 train images, 1681 validation images and 839 test images, offered to develop a robust system that is tested and generalized on different scenes, at a 640×640 resolution.

Furthermore, the dataset has annotations that includes three classes: ‘fire’ (flame regions), ‘smoke’ (particulate plumes), and ‘other’ (background/nuisances like steam). Bounding boxes for each image were provided in YOLO format (.txt files), containing “class_id”, “center_x”, “center_y”, “width” and “length” of each box/detected smoke or fire, in that particular image.

3.3. ML/DL Models

During performance evaluation, various models were used, such as the aforementioned sixteen (16) classifiers, ensemble methods, and DL for sensor-based binary classification. The goal was to leverage lightweight architectures suitable for edge inference.

Model Architectures and Training

Models were implemented via scikit-learn library pipelines (with StandardScaler for non-tree-based learners) and custom TF-Keras for NNs.

- Ensemble: It included models such as Linear (LogisticRegression, RidgeClassifier), Non-linear (KNeighborsClassifier, SVC), Trees (DecisionTreeClassifier, ExtraTreeClassifier, RandomForestClassifier), Boosting (GradientBoostingClassifier, AdaBoostClassifier, HistGradientBoostingClassifier, XGBoost, LightGBM, CatBoost), and Neural (MLPClassifier, TF-Keras NN).
- NN Configuration: The TF-Keras model used a sequential architecture (64-32-16 ReLU layers with 20% Dropout), optimized with the Adam optimizer ($\text{lr} = 0.001$), binary cross-entropy loss, and early stopping ($\text{val_loss patience} = 5$).

Training used the full train split ($n = 50,103$). For the TF-Keras NN, data was scaled and fitted with $\text{validation_split} = 0.2$, $\text{shuffle} = \text{True}$, $\text{epochs} = 50$, and $\text{batch_size} = 32$. We utilized HPO for XGBoost with Optuna (20 trials, TPE sampler), optimizing parameters like $n_estimators$ [100–600], maxdepth [3–20], and learning_rate [0.01–0.3] by minimizing negative CV-F1 (3-fold StratifiedKFold).

3.4. Evaluation and Explainable AI (XAI)

Models were assessed on the held-out test set ($n = 12,526$) using accuracy, precision, recall, F1-score (macro-averaged), and AUC-ROC. Training time was timed via the time library. In addition, the top-F1 model was serialized via joblib (for sklearn) or Keras save (for NN). Post hoc explainability used SHAP (v0.46.0) on the best model, subsetted to 1,000 test instances. Explainers were model-specific: GradientExplainer (KerasNN), LinearExplainer (linear models), TreeExplainer (trees/boosters), and KernelExplainer (fallback). Finally, mean absolute SHAP values were used to yield feature importances.

3.5. Vision-Based Detection Pipeline

As far as object vision detection is concerned, we decided to compare a number of YOLO nano variants from Ultralytics, starting with older versions (v5n, v8n) and continuing with some modern ones (v10n, v11n, v12n), fine-tuning them using the dataset for multi-class detection (‘fire’, ‘other’, ‘smoke’) and prioritizing sub-10 GFLOPs for edge viability.

3.5.1. Training

The training process for any YOLO object detection model (such as the Nano versions of v5n, v8n, v11n, or v12n) is based on transfer learning and fine-tuning a powerful pre-trained network for a specialized task. This involves loading generic weights, often pre-trained on the massive COCO dataset to recognize thousands of everyday objects, and then retraining the model using our much smaller, specific dataset. In our case, the dataset is organized with training images in `Detection_Dataset/train/images`, validation images in `Detection_Dataset/valid/images`, and testing images in `Detection_Dataset/test/images`, and the classes are defined in the configuration file (`data.yaml`) as 'fire', 'other', and 'smoke'. The training loop runs for 20 epochs (full passes over the dataset), where the model continually adjusts its weights based on the calculated loss (the error between its predictions and the ground truth bounding boxes and labels) using an optimizer like Stochastic Gradient Descent. Images are resized to a preset input size of 640×640 pixels to ensure consistency, and a fixed random seed (`seed = 42`) is used to make the training process reproducible. Additionally, the configuration links the dataset to a Roboflow workspace (`new2firedataset`) and project (`newfire2`) for easier dataset management and versioning. The goal of this setup is to efficiently leverage the network's existing ability to extract general visual features while fine-tuning the final layers to precisely identify our custom classes. The final saved model weights (`best.pt`) are therefore highly accurate and robust for real-time detection of fire and smoke scenarios. Finally, the epochs are set to 20 to challenge YOLO models' capabilities and limits, balancing training efficiency with the demonstration of model performance on lightweight architectures.

3.5.2. Testing

The trained YOLO model is tested on a live video stream from a camera by running the core inference process in a continuous loop. This testing procedure involves four key steps performed for every frame: First, the system initializes the model by loading the best-performing weights (`best.pt`). Second, it captures a frame from the default camera source (`cv2.VideoCapture(0, cv2.CAP_DSHOW)`). Third, the loaded model then runs a real-time prediction on the captured frame, using parameters like a minimum confidence threshold (e.g., `conf = 0.5`) and the standardized image size (`imgsz = 640`). Ultimately, the results which include the predicted class labels ('fire' or 'smoke') and the corresponding bounding box coordinates are immediately rendered back onto the live video feed using the model's plotting function. This generates an annotated frame displayed to the user, allowing for instant, visual verification of the model's ability to accurately detect and localize fire and smoke in a real-world environment.

3.5.3. Explainable AI for Vision Models

Testing and explanation of the trained model using LIME are performed to understand why the YOLO model made a specific 'fire' or 'smoke' detection on a single image. This methodology does not run on a live stream but rather on a single, representative test image. More precisely, to generate the LIME explanation, the method works by perturbing the image, thus generating hundreds of slightly modified versions of the original image by hiding or masking different super-pixel regions. Afterwards, each modified image is fed to the trained YOLO model and the prediction scores for the target class (e.g., 'fire' or 'smoke') are recorded. Finally, we continued with Local Model Fitting, where LIME then trains a simple, interpretable linear model that weighs how much each visible super-pixel contributes to the final prediction score. The output is a heatmap, a colored overlay that highlights the specific pixels and regions of the image that the model was focusing on

to make its final classification. For instance, our script separately visualizes the regions influencing a ‘fire’ prediction (in red) and a ‘smoke’ prediction (in blue).

3.6. Explainable Decision-Only Sensor Fusion Framework

The proposed framework follows a decision-level (late) fusion paradigm, specifically designed to combine heterogeneous information from environmental sensors and DL-based visual detection models. Rather than employing end-to-end multimodal learning, the system processes sensor data and RGB images through independently trained models optimized for their respective modalities. By integrating these probabilistic outputs through a weighted fusion mechanism, the architecture exploits the complementary nature of each system, capturing different evidence of fire activity to produce reliable, low-false-alarm alerts while effectively mitigating modality-specific failure modes.

Moreover, the tabular sensors (such as CO₂, VOC, particulate matter, temperature, and humidity) provide continuous, stable measurements of environmental conditions. These sensors respond to the chemical and thermal signatures that typically precede or accompany combustion. However, in industrial settings, they can also be influenced by benign processes like welding, steam release, dust, and other noise factors, creating uncertain situations. For this reason, their output is normalized and passed through an ML classifier to generate a probabilistic “sensor-side fire likelihood,” noted as P_{sensor} . This probability expresses how closely the current sensor pattern resembles combustion-like conditions.

Additionally, the visual detection pathway uses a YOLO model specifically trained to detect flames and smoke. YOLO offers strong specificity because it relies on direct visual evidence of combustion phenomena. Yet, in factories, lighting variations, reflections from metal surfaces, motion blur, or airborne particulates can create misleading visual cues, leading to false alarms. To manage this, the model extracts a confidence score for fire and smoke, noted as $P_{\text{yolo_fire}}$ and $P_{\text{yolo_smoke}}$, each treated as separate probability sources contributing to the final decision. To unify both sensing streams, the framework applies a weighted-multiplicative fusion rule. The normalized sensor probability and YOLO probability are each raised to their respective importance weights, 55% for the sensors and 45% for the visual detector, and then multiplied together. Mathematically, the fused probability is described in Formula (1).

Also, the choice of a weighted-multiplicative rule is theoretically grounded in Bayesian Joint Probability for independent observers. Unlike additive fusion, which may allow a single high-confidence modality to mask a low-confidence one, the product rule effectively acts as a probabilistic ‘AND’ gate. This ensures that the final alert is a result of rigorous cross-verification, mathematically penalizing the fusion score if either modality lacks evidence of combustion. This approach is critical for prioritizing precision in high-noise industrial environments.

Ultimately, the system also incorporates a high-confidence visual override mechanism for rapid response. If YOLO detects fire or smoke with very high confidence, specifically $P_{\text{yolo}} \geq 0.85$, and the sensors meet the minimum abnormality requirement ($P_{\text{sensor}} \geq 0.65$), the framework triggers an alert even if P_{fusion} is below the main confirmation threshold. This mechanism compensates for situations where flames are visually obvious but environmental sensors are delayed due to airflow patterns, dilution, outdoor conditions, or large factory volumes and distances.

Adaptive Learning and Dynamic Weighting

While the current framework utilizes static weights ($W_s = 0.55$, $W_v = 0.45$) to establish a baseline for logical validation, the modular architecture is designed to support future Adaptive Learning Strategies as well. By decoupling the decision logic from the feature

extraction, the system can provide a foundation for Dynamic Weight Adjustment based on environmental metadata.

Future iterations of the framework could employ a Gating Network to modify weights in real time. For instance, the system could automatically decrease the visual weight (W_v) during low-light conditions or instances of extreme lens obstruction (e.g., steam or dust), while simultaneously increasing the sensor weight (W_s) to maintain reliability. Conversely, in high-airflow scenarios where chemical dilution is likely, the visual priority can be adaptively increased. This flexibility transitions the system from a fixed heuristic model to an intelligent, self-optimizing agent capable of maintaining high sensitivity and low false-alarm rates across shifting industrial contexts.

3.7. Fusion Dataset Construction and State-Space Alignment

To be able to validate the proposed decision-level fusion framework, a unified fusion dataset was constructed using a State-Space Alignment strategy. While the environmental sensor data and visual frames originate from independent high-quality datasets, they are integrated at the decision level to represent a comprehensive matrix of industrial combustion states. This approach is specifically chosen for the decision-level (late) fusion system, where the primary objective here is to evaluate the logical robustness of the fusion policy rather than the raw signal correlation.

Moreover, the dataset was synthesized by synchronizing 10,000 iterations of real-time visual streams with stratified sensor samples (5000 non-alarm and 5000 alarm states). By intentionally decoupling the modalities during the synthesis phase, the dataset includes “adversarial” states, such as high visual confidence paired with baseline sensor readings, which are essential for testing the probabilistic “AND” gate logic. This ensures that the framework learns to prioritize cross-modal verification, effectively simulating real-world scenarios where one modality may experience a failure mode (e.g., optical reflections or sensor transport lag).

Furthermore, this construction allows for Policy Distillation, where the ML model is trained to internalize the complex fusion rules, including the high-confidence visual override. This methodology confirms that the system’s response is deterministic and logically consistent across the entire operational state-space, providing a foundation for safety-critical deployments where exact temporal synchronization between heterogeneous hardware is often unfeasible.

4. Results

In this section, results are presented regarding the dual-modality detection framework, beginning with the performance and explainability of the sensor-based classification system, evaluated with various metrics, and then followed by the performance evaluation of the vision-based YOLO object detection nano models. Some of the most crucial metrics used for evaluation are Precision, Recall and F1 Score, which are presented below:

- Precision = $TP / (TP + FP)$, with TP: True Positive and FP: False Positive;
- Recall = $TP / (TP + FN)$, with TP: True Positive and FN: False Negatives;
- F1 Score = $(2 \times ((Precision \times Recall) / (Precision + Recall)))$

4.1. Performance Analysis of Tabular Classification Models

We conducted performance evaluation across 16 production-ready ML and DL classifiers to identify the optimal model for the sensor-based system (P_{sensor}). The high-quality and low-noise nature of the sensor data resulted in accurate performance across all tree-based and ensemble methods, with multiple models achieving maximum F1-Scores.

4.1.1. Model Benchmarking and Comparison

Performance results on the held-out test set are summarized below in Table 1, ranked by the macro-averaged F1-Score and training time of the models.

Table 1. Performance metrics of the ML and DL models.

Model	Accuracy	Precision	Recall	F1 Score	AUC Score	Training Time
Decision Tree	1.0	1.0	1.0	1.0	1.0	0.283
LightGBM	1.0	1.0	1.0	1.0	1.0	0.512
RandomForest	1.0	1.0	1.0	1.0	1.0	6.329
XGBoost	1.0	1.0	1.0	1.0	1.0	0.34
HistGradientBoosting	1.0	1.0	1.0	1.0	1.0	1.895
CatBoost	1.0	1.0	1.0	1.0	1.0	13.377
AdaBoost	1.0	1.0	1.0	1.0	1.0	4.881
ExtraTree	1.0	1.0	1.0	1.0	0.999	0.03
GradientBoosting	1.0	1.0	1.0	1.0	1.0	16.585
KNN	0.998	0.998	0.999	0.999	0.999	2.338
KerasNN	0.997	0.996	1.0	0.998	1.0	63.78
SVC	0.968	0.964	0.992	0.978	0.997	150.556
MLPClassifier	0.965	0.956	0.997	0.976	0.994	4.943
LogisticRegression	0.895	0.909	0.948	0.928	0.964	0.418
RidgeClassifier	0.871	0.857	0.983	0.916	NaN	0.054
GaussianNB	0.768	0.765	0.975	0.858	0.936	0.073
Dummy	0.715	0.715	1.0	0.834	0.5	0.015

4.1.2. Key Findings and Model Selection

Multiple models, including the LightGBM, and DecisionTreeClassifier, achieved perfect F1-Scores (1.0000) and decent real-time processing training times. These results shifted the final selection criteria to model robustness and deployment overhead. On the other hand, it is clearly illustrated that complex models like NN or SVC require more training time (63.78 s and 150.556 s, respectively) because of their architectures, computation load and data-hungry features, making them unsuited for real-time processes in constrained environments with limited amount of data.

The LightGBM model was ultimately selected as the optimal model for the final P_{sensor} component within the hybrid system, achieving practically perfect prediction levels as shown below on Figure 1. While the single Decision Tree model achieved a perfect score of 1.0000 across all metrics (Accuracy, F1-score, and AUC), it presented a critical risk of overfitting, memorizing the specifics of the training data and leading to potential instability when deployed with noisy, real-world sensor readings. The LightGBM model provided virtually identical, perfect performance (e.g., 1.0000 F1-score and 1.0000 AUC), but with superior robustness and regularization, providing more reliable results. This architectural choice ensures significantly better generalization capabilities to new unseen data as well.

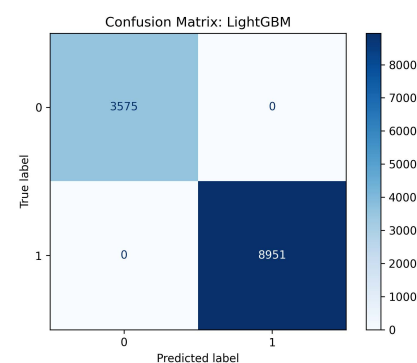


Figure 1. Confusion matrix results of the LightGBM ML model.

4.2. Explainable AI Analysis of RandomForestClassifier (SHAP)

A post hoc SHAP analysis was performed on the selected LightGBM model to ensure transparency and validate the physical relevance of its decision-making process. The results, reflecting the mean absolute contribution of each feature to the model's output probability, are presented below in Table 2.

Table 2. Features' contribution importance to the model's final decision-making process.

Feature	Importance
Pressure	4.988
TVOC	3.348
PM1.0	1.017
Humidity	0.848
Temperature	0.513
Raw H2	0.491
Raw Ethanol	0.385
NC0.5	0.252
PM2.5	0.018
ECO2	0.008
NC2.5	0.004
NC1.0	0.003

4.2.1. Feature Importance Ranking

The SHAP analysis presented below on Figure 2, produced the following definitive feature importance ranking for the LightGBM model:

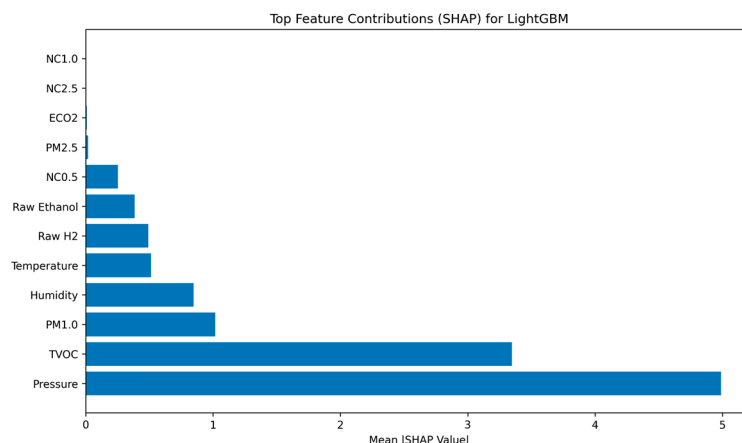


Figure 2. Bars showcasing features' contribution importance to the model's final decision-making process.

4.2.2. XAI Interpretation

The SHAP ranking confirms that the model's decision-making is logically sound and aligned with the physical phenomenology of a fire event. Pressure and TVOC (Total Volatile Organic Compounds) are the dominant features, confirming that the model effectively integrates specialized chemical signatures and associated atmospheric changes. The simultaneous high ranking of these two distinct sensor types contributes significantly to the system's low false-positive rate. Metrics commonly associated with environmental noise, such as Temperature (Rank 5) and eCO₂ (Rank 10), are correctly de-prioritized, demonstrating that the classifier relies on complex chemical and pressure patterns rather than simple thermal triggers.

4.3. Comparative Analysis of Vision-Based Detection Nano Models (YOLO Benchmark)

This section details the comparative performance of the five selected YOLO nano variants (v5n, v8n, v10n, v11n, v12n) on the custom Fire and Smoke Image-Label Dataset. The evaluation focuses on the critical trade-off between localization accuracy (mAP) and real-time efficiency (Inference Speed) for edge deployment.

4.3.1. YOLOv5nu Analysis

The YOLOv5nu nano variant establishes a critical baseline, providing a strong balance between localization accuracy and its resource profile, as presented below on Figures 3–7. The model achieved a competitive mAP@0.50 of 0.8267 and a satisfactory mAP@0.50-0.95 of 0.5422. These values confirm its high reliability in identifying and accurately bounding both fire and smoke objects across moderate Intersection over Union (IoU > 0.5) and stricter (IoU > 0.95) thresholds.

For real-time industrial deployment, YOLOv5nu provides an inference speed of 8.5 to 10.5 FPS (frames per second). While this speed exceeds the minimum required throughput for near real-time monitoring, it leaves limited overhead for more demanding hardware or higher video resolutions. With a compact architecture of 2.6 million parameters, YOLOv5nu is lightweight, but the newer, more optimized architectures have superior speed-to-accuracy trade-offs.

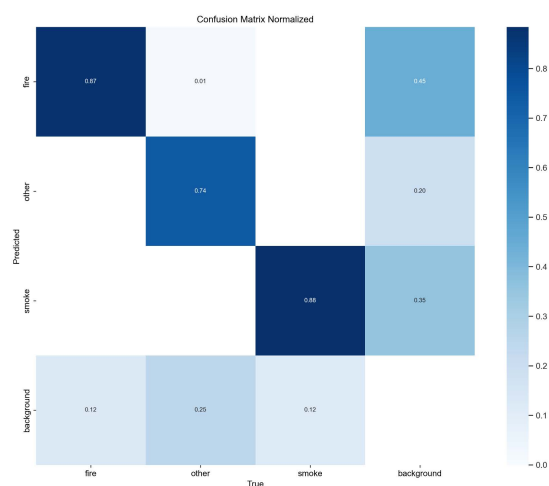


Figure 3. Normalized confusion matrix of smoke/fire detection using YOLOv5nu.

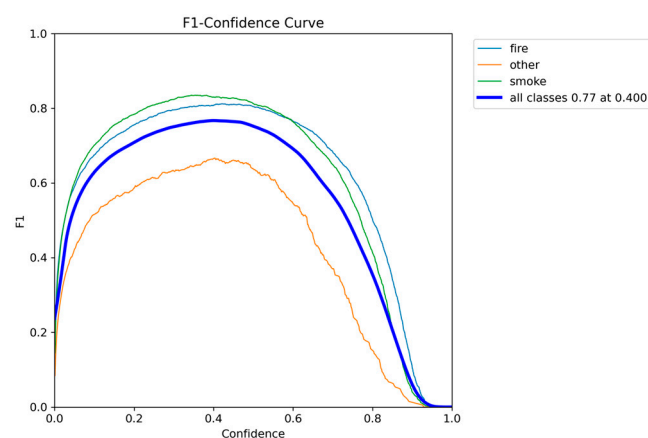


Figure 4. F1-Confidence metric performance results of YOLOv5nu.

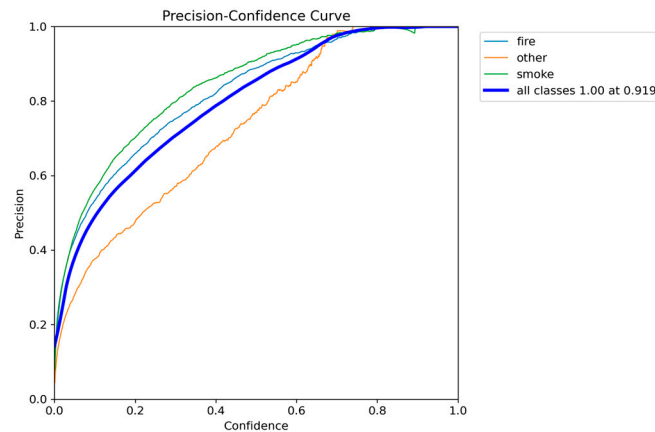


Figure 5. Precision–Confidence performance results of YOLOv5nu.

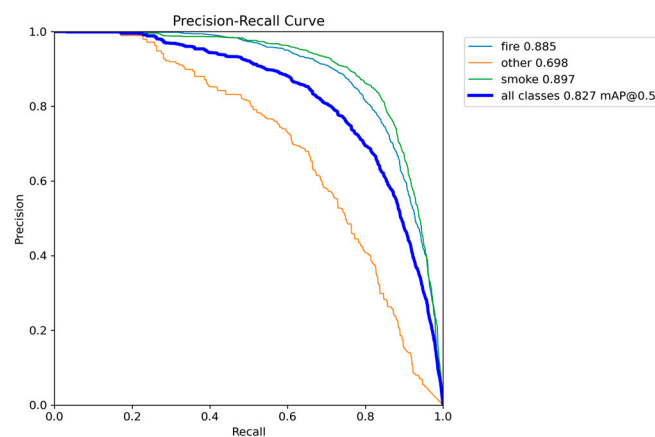


Figure 6. Precision–Recall performance results of YOLOv5nu.

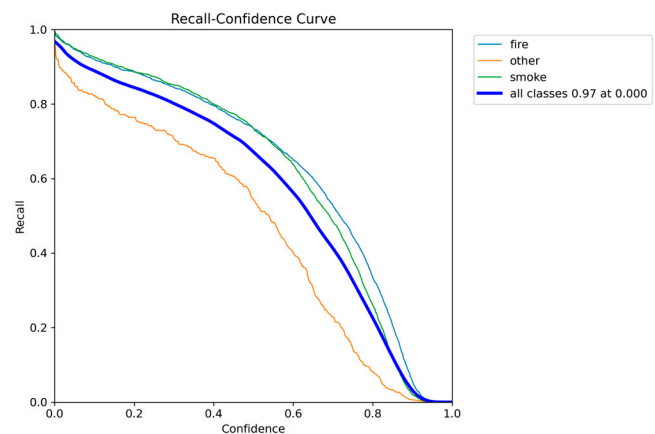


Figure 7. Recall–Confidence performance results of YOLOv5nu.

In addition, performance analysis of the training metrics (including train/box_loss, val/cls_loss, and epoch-based precision/recall) as shown below on Figure 8, confirmed the model's stable and rapid convergence. The consistent decrease in loss across both training and validation sets, coupled with the monotonic increase in epoch-based precision and recall, indicates that the model did not suffer from significant overfitting or underfitting. This stability validates the model's robustness and the effectiveness of the chosen training configuration.

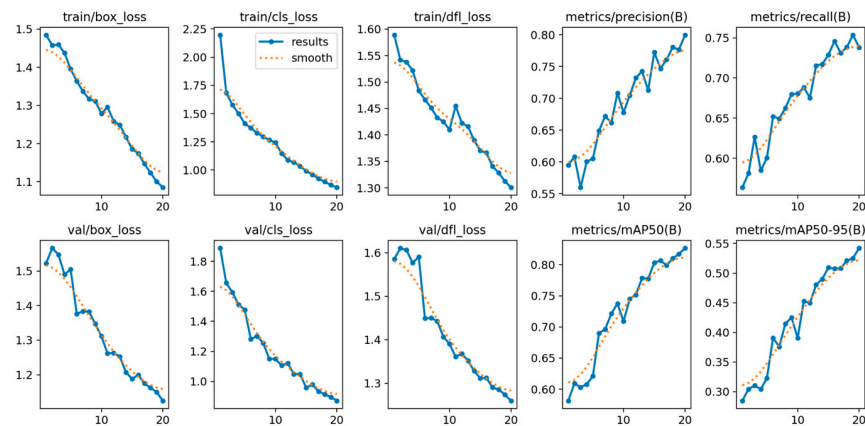


Figure 8. Different metrics' performance results of YOLOv5nu.

Below, Figure 9 illustrates the detections of the YOLOv5nu model, which can detect from small to larger smoke and fire regions successfully.



Figure 9. Detection and bounding boxing of smoke/fire regions with YOLOv5nu.

4.3.2. YOLOv8n Analysis

The YOLOv8n model presents additional gains in accuracy while maintaining a constrained resource footprint. It achieved the best overall performance, obtaining the highest mAP@0.50 of 0.8320 and a marginal increase, as well as second-best in comprehensive localization accuracy, with mAP@0.50-0.95 reaching 0.5489. This indicates a minor increase in the model's ability to precisely localize fire and smoke objects, particularly at stricter IoU thresholds, as shown below on Figures 10–14.

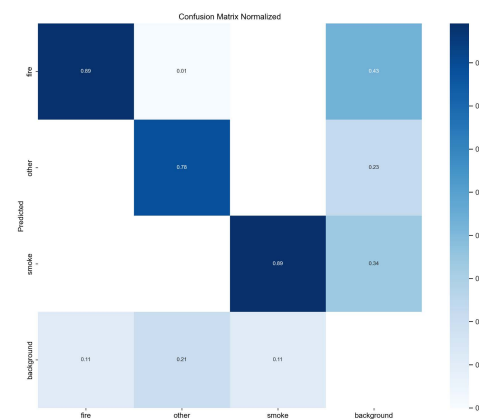


Figure 10. Normalized confusion matrix of smoke/fire detection using YOLOv8n.

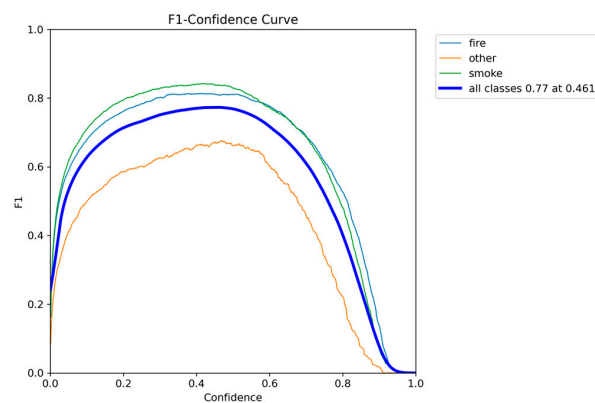


Figure 11. F1–Confidence performance results of YOLOv8n.

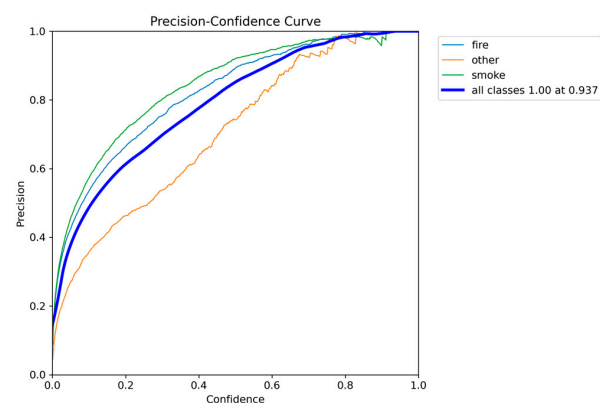


Figure 12. Precision–Confidence performance results of YOLOv8n.

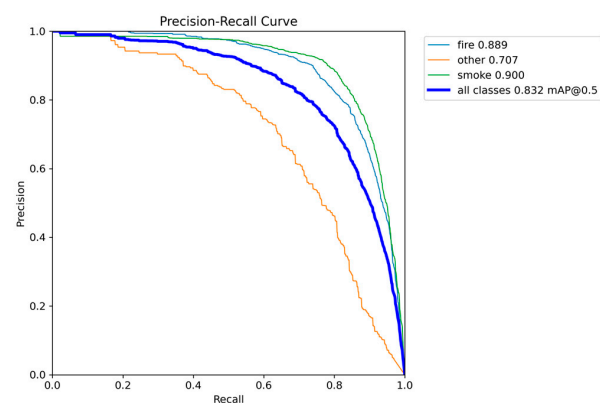


Figure 13. Precision–Recall performance results of YOLOv8n.

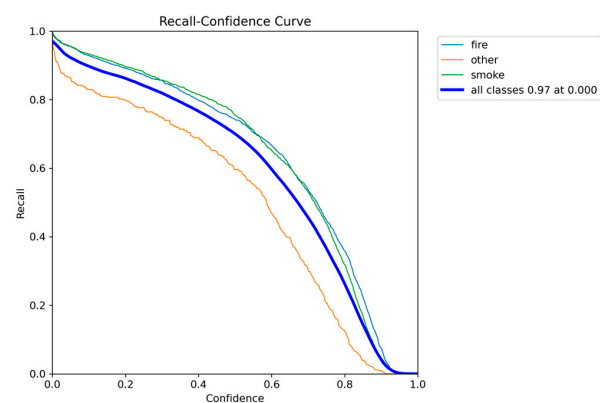


Figure 14. Recall–Confidence performance results of YOLOv8n.

However, in terms of efficiency, YOLOv8n exhibited a slight performance degradation compared to YOLOv5n, achieving an inference speed of 7.5 to 9.5 FPS. While this speed still meets the minimum real-time requirement for the edge device, it is slower than the v5nu baseline, but still the second-best one, among the YOLO nano models. The model's complexity increased to 3.2 million parameters, making it slightly larger. This suggests that the architectural improvements place YOLOv8n among nano-scale YOLO variants as the most favorable trade-off, between detection accuracy and inference speed, making it the most suitable model for real-time smoke and fire detection in resource-constrained edge devices.

Finally, the analysis of the YOLOv8n training metrics (such as the bounding box loss, which finalized at approximately 1.1117), presented below on Figure 15, confirmed highly stable and robust learning. The smooth, consistent decrease in loss across both the training and validation sets indicates that the model converged effectively without significant oscillation or divergence.

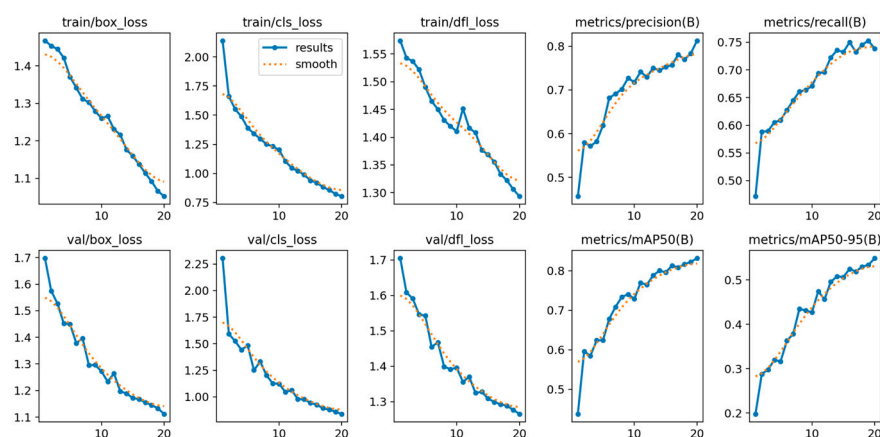


Figure 15. Different metrics' performance results of YOLOv8n.

Below, Figure 16 illustrates the detections of the YOLOv8n model, which can detect from small to larger smoke and fire regions successfully.



Figure 16. Detection and bounding boxing of smoke/fire regions with YOLOv8n.

4.3.3. YOLOv10n Analysis

The YOLOv10n model presents a noticeable shift in the accuracy–speed trade-off compared to its predecessors. It is the most lightweight model so far, with a size of 2.3 million parameters, making it highly attractive for the most resource-constrained devices. It also maintains competitive inference speed, ranging from 6.9 to 9.5 FPS. This

speed means that while it is highly compact, its real-time performance is variable and can drop below the desired seven FPS threshold, posing a reliability risk.

In terms of accuracy, YOLOv10n achieved an mAP@0.50 of 0.7906 and an mAP@0.50-0.95 of 0.5156. While these scores are robust, they represent a decline in performance compared to both YOLOv5n and YOLOv8n, as shown below on Figures 17–21. This decline suggests that the architectural changes focused on reducing the parameter count finally have an impact on the model's ability to maximize detection accuracy of the fire and smoke dataset.

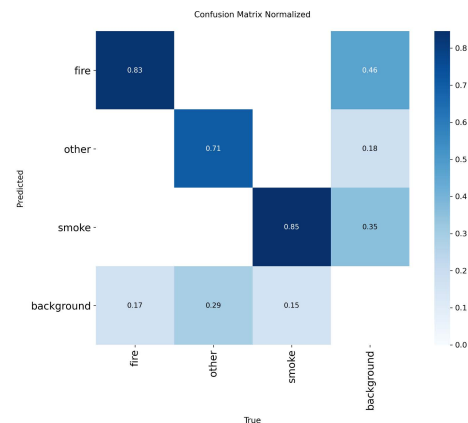


Figure 17. Normalized confusion matrix of smoke/fire detection using YOLOv10n.

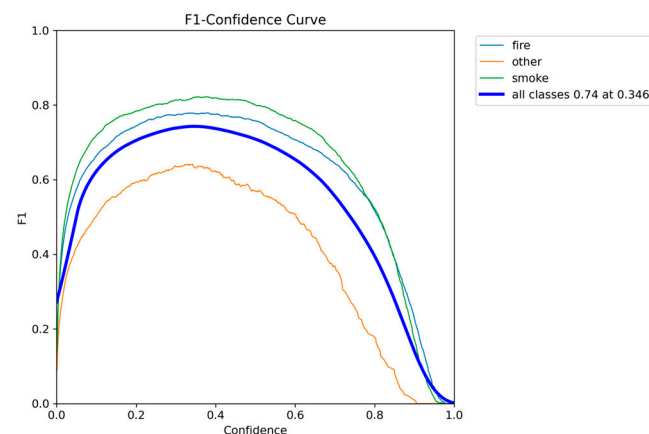


Figure 18. F1–Confidence performance results of YOLOv10n.

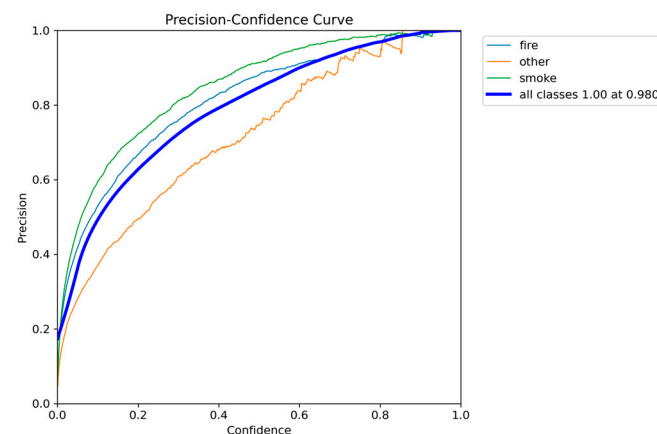


Figure 19. Precision–Confidence performance results of YOLOv10n.

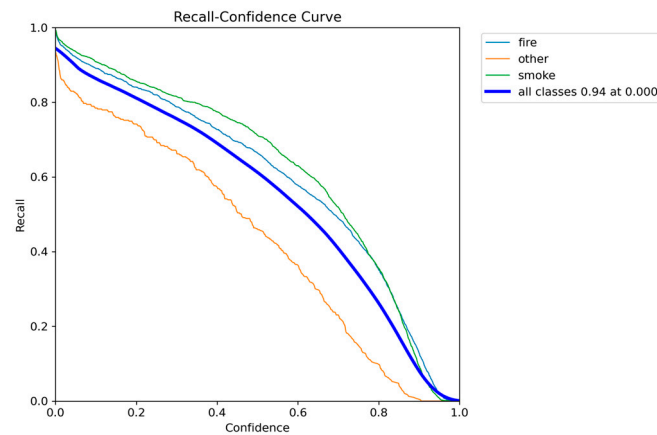


Figure 20. Recall–Confidence performance results of YOLOv10n.

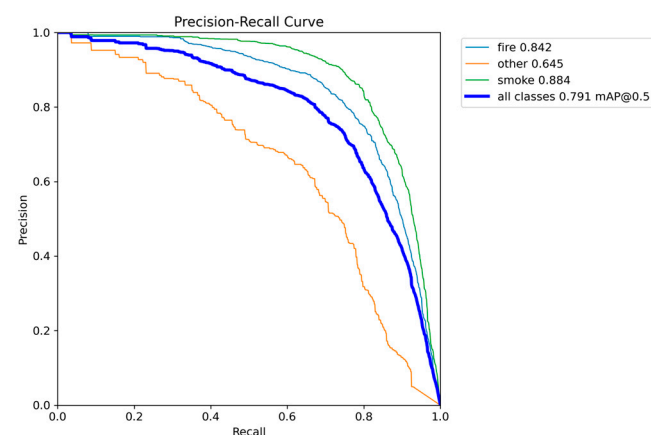


Figure 21. Precision–Recall performance results of YOLOv10n.

The training metrics confirm that the YOLOv10n model successfully converged, indicated by the low difference between the final training box loss (approximately 2.17) and the validation box loss (approximately 2.24), as presented below on Figure 22. This minimal loss gap suggests that despite the architectural modifications for size reduction, the model trained stably without significant signs of overfitting, confirming the reliability of the reported accuracy figures.

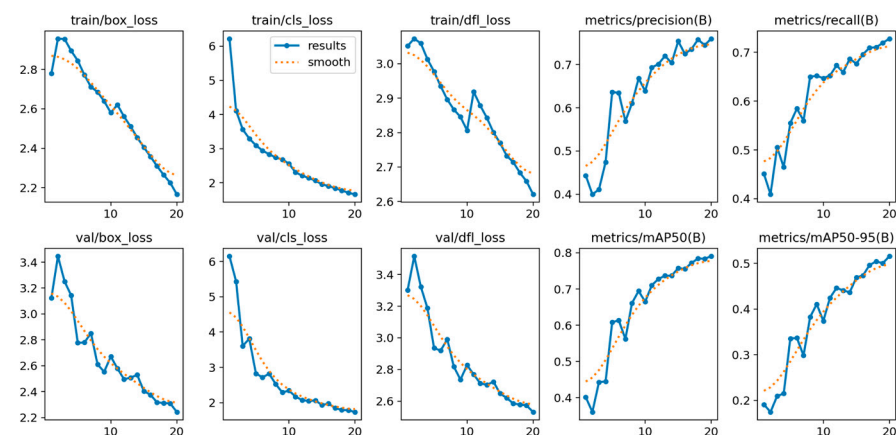


Figure 22. Different metrics' performance results of YOLOv10n.

Below, Figure 23 illustrates the detections of the YOLOv10n model, which can detect from small to larger smoke and fire regions successfully.



Figure 23. Detection and bounding boxing of smoke/fire regions with YOLOv10n.

4.3.4. YOLOv11n Analysis

The YOLOv11n model achieved an mAP@0.50 of 0.8293 and an mAP@0.50-0.95 of 0.5474. These scores are nearly identical to the high accuracy achieved by the YOLOv8n model, significantly outperforming the low-accuracy YOLOv10n variant. In terms of efficiency and accuracy, as presented below on Figures 24–28, YOLOv11n achieved a favorable parameter count of 2.6 million, which matches the compact size of YOLOv5n and is smaller than YOLOv8n. Its inference speed of 7.0 to 8.9 FPS is stable and consistently meets the real-time threshold (7 FPS), avoiding the risk of performance drops observed in YOLOv10n.

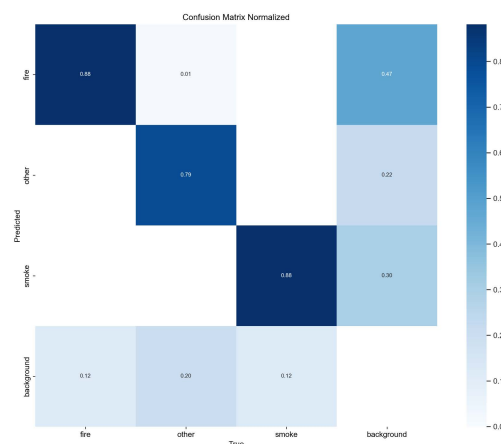


Figure 24. Normalized confusion matrix of smoke/fire detection using YOLOv11n.

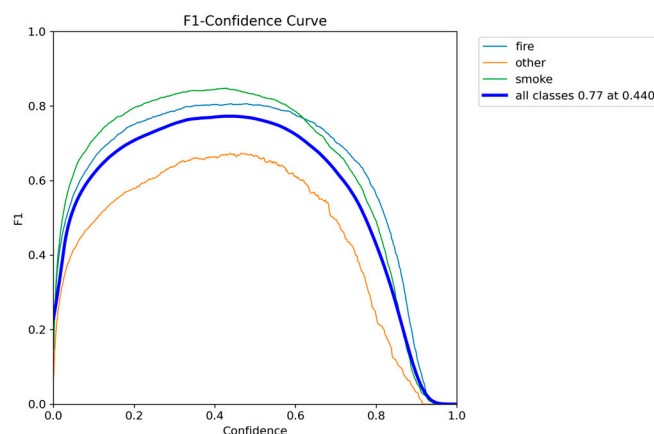


Figure 25. F1–Confidence performance results of YOLOv11n.

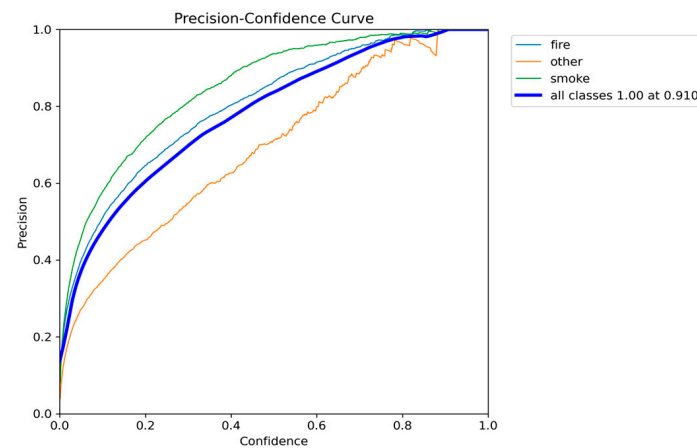


Figure 26. Precision–Confidence performance results of YOLOv11n.

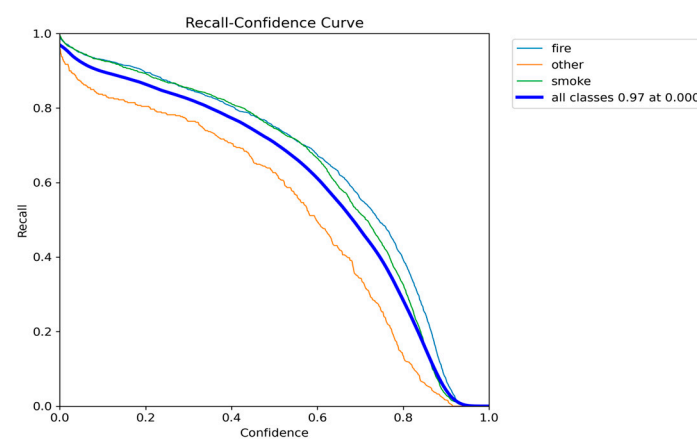


Figure 27. Recall–Confidence performance results of YOLOv11n.

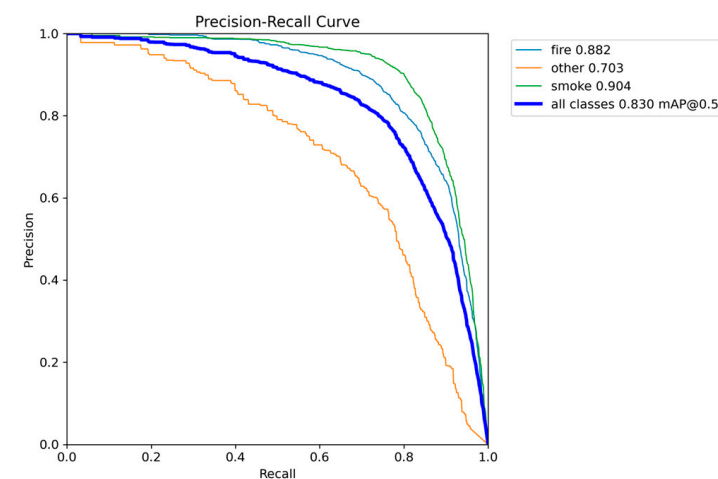


Figure 28. Precision–Recall performance results of YOLOv11n.

The final analysis of the YOLOv11n training data confirms excellent convergence stability. The final training box loss (approximately 1.06) and validation box loss (approximately 1.09) are both very low and tightly coupled, as shown below on Figure 29.

Below, Figure 30 illustrates the detections of the YOLOv11n model, which can detect from small to larger smoke and fire regions successfully.

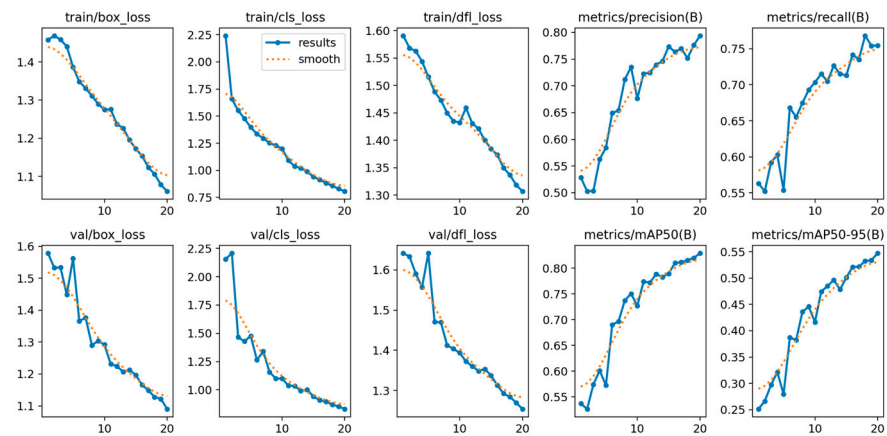


Figure 29. Different metrics' performance results of YOLOv11n.



Figure 30. Detection and bounding boxing of smoke/fire regions with YOLOv11n.

4.3.5. YOLOv12n Analysis

The YOLOv12n model demonstrates the highest overall localization accuracy among all benchmarked nano variants, achieving an mAP@0.50-0.95 score of 0.5544 and the second-best mAP@0.50 of 0.8306. This high mAP@0.50-0.95 value indicates its superior ability to place highly precise bounding boxes around fire and smoke, as presented below on Figures 31–35, even at very strict IoU thresholds. The model maintains a compact structure of 2.6 million parameters, matching YOLOv5n and YOLOv11n in size.

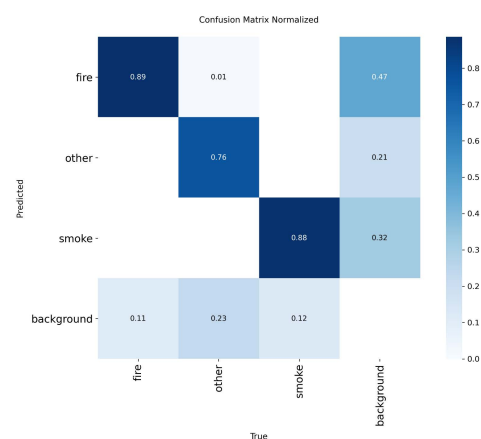


Figure 31. Normalized confusion matrix of smoke/fire detection using YOLOv12n.

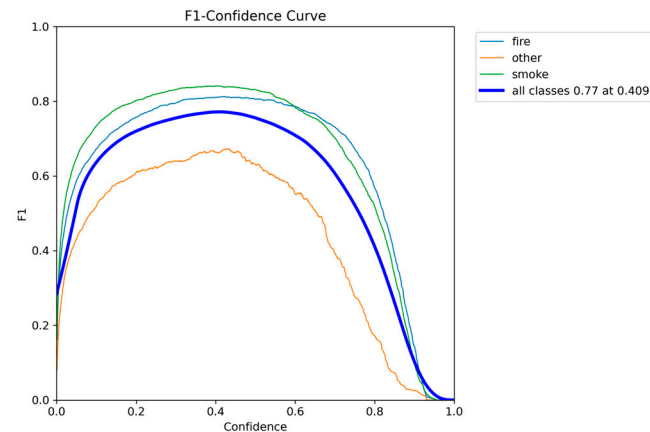


Figure 32. F1–Confidence performance results of YOLOv12n.

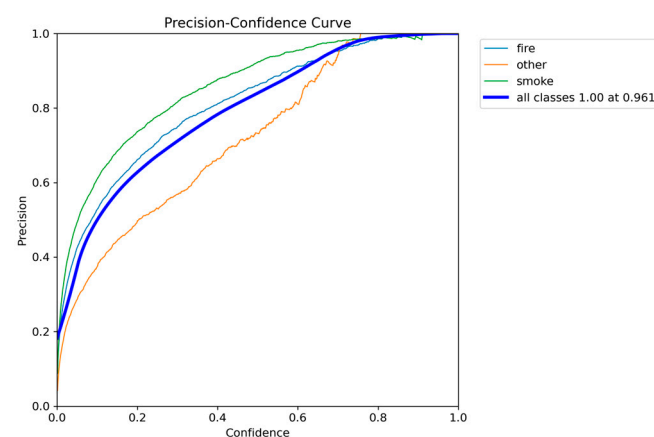


Figure 33. Precision–Confidence performance results of YOLOv12n.

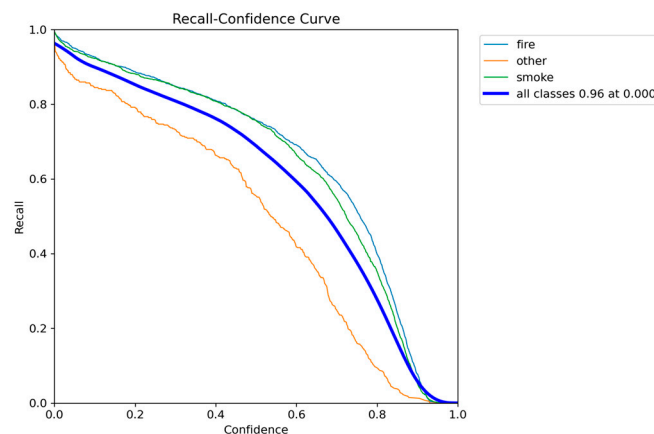


Figure 34. Recall–Confidence performance results of YOLOv12n.

However, YOLOv12n showed the slowest inference speed of 6.2 to 7.3 FPS. Since the minimum real-time requirement for the target edge device is seven FPS, this model is almost unreliable for true real-time deployment in our hardware edge device, as its performance frequently drops below the critical threshold.

The training metrics for YOLOv12n confirm exceptional learning optimization. The final training box loss (approximately 1.05) and validation box loss (approximately 1.09) are extremely low and tightly matched, as shown below on Figure 36, signifying near-perfect convergence and superb generalization capability. This stability validates the model's accuracy but confirms that its performance limitation lies strictly in its architectural complexity during inference.

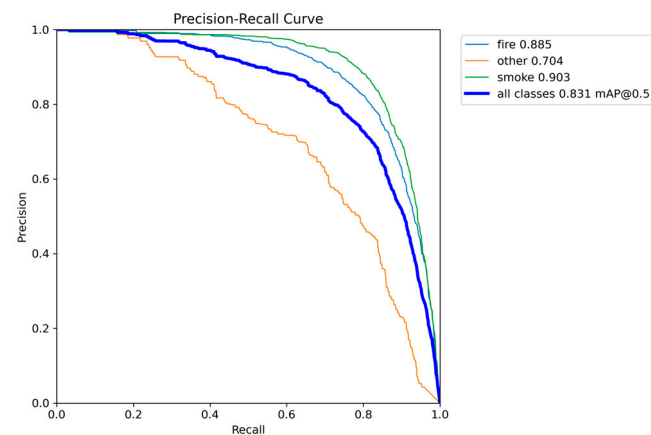


Figure 35. Precision–Recall performance results of YOLOv12n.

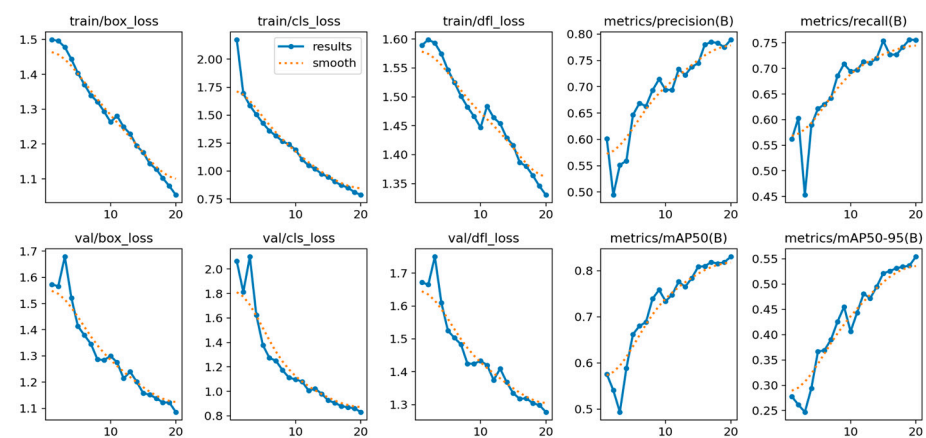


Figure 36. Different metrics' performance results of YOLOv12n.

Below, Figure 37 illustrates the detections of the YOLOv12n model, which can detect from small to larger smoke and fire regions successfully.



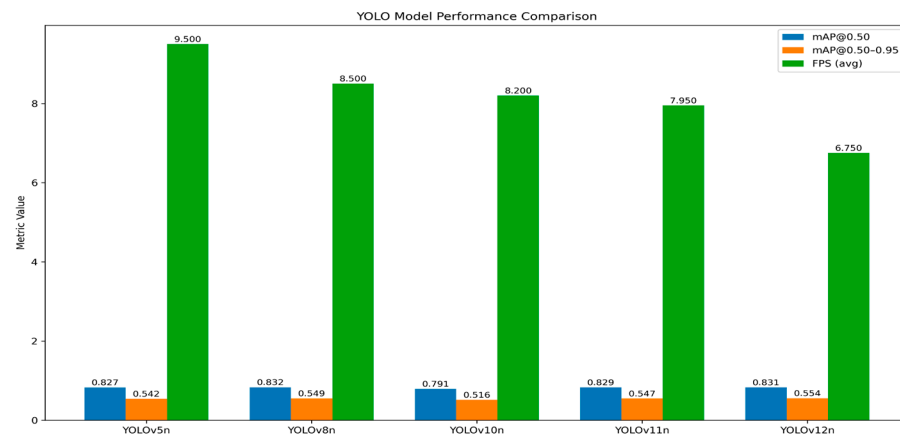
Figure 37. Detection and bounding boxing of smoke/fire regions with YOLOv12n.

4.3.6. Comparative Analysis and Optimal Model Selection

The comparative analysis of the five YOLO nano models reveals a clear trade-off spectrum between accuracy, model size, and real-time inference speed, which is summarized below on Table 3 and Figure 38 as follows:

Table 3. Presenting the final mAP@0.50, mAP@0.50-0.95 and FPS performance results for each YOLO nano model.

Model	mAP@0.50	mAP@0.50-0.95	Inference Speed (FPS)	Avg FPS	Parameters (M)
YOLOv5	0.8267	0.5422	8.5–10.5	9.5	2.6
YOLOv8	0.8320	0.5489	7.5–9.5	8.5	3.2
YOLOv10	0.7906	0.5156	6.9–9.5	8.2	2.3
YOLOv11	0.8293	0.5474	7.0–8.9	7.95	2.6
YOLOv12	0.8306	0.5544	6.2–7.3	6.75	2.6

**Figure 38.** Presenting the final mAP@0.50, mAP@0.50-0.95 and FPS performance results for each YOLO nano model.

Models like YOLOv12n achieved the highest overall localization accuracy with an mAP@0.50-0.95 of 0.5544, but its slow inference speed (6.2 to 7.3 FPS) makes it unreliable for the target seven FPS real-time deployment threshold. Conversely, YOLOv10n was the most compact (2.3 million parameters) but compromised too much on accuracy, achieving the lowest mAP@0.50 of 0.7906. YOLOv11n provided a strong balance with high accuracy (0.8293 mAP@0.50) and compact 2.6 million parameters, but its speed (minimum 7.0 FPS) was too close to the critical limit. Although YOLOv5n achieved marginally higher inference speed [8.5–10.5 FPS] and decent mAP@0.50 of 0.8267 and mAP@0.50-0.95 of 0.5422, given the necessity for robust, high-speed performance on a lightweight edge device, YOLOv8n was selected as the optimal model, providing the best and excellent detection accuracy (0.8320 mAP@0.50) and the second-best mAP@0.50-0.95 of 0.5489 with a compact size (3.2 million parameters), as well as a decent and stable inference speed (ranging from 7.5 to 9.5 FPS).

4.4. Explainable AI (XAI) Analysis of Detection Models

In order to provide insight into the detection mechanism of the chosen models and validate their decision-making process, LIME was employed as the post hoc eXplainable AI technique, applied on images. This analysis was systematically performed on each YOLO nano variant (YOLOv5n, v8n, v10n, v11n, and v12n), utilizing the exact same set of nine test images [45–53] for every model to ensure a fair, direct comparison of interpretability. LIME works by approximating the complex model's prediction locally, visually isolating the specific input features (pixels) that most contributed to the model's output. The resulting heatmaps highlight the most contributing area for detection with a distinct color coding applied: areas contributing to the 'fire' class (Class 0) were highlighted in blue, and areas contributing to the 'smoke' class (Class 2) were highlighted in red.

4.4.1. YOLOv5nu

The analysis of the YOLOv5n LIME superplot reveals a highly focused and efficient feature selection strategy across the nine test images. The LIME heatmaps, which highlight features contributing to the ‘fire’ class in blue and the ‘smoke’ class in red, as shown below on Figure 39, demonstrate a tight concentration of contributing pixels precisely over the core regions of the detected objects. The model presents minimal diffusion of feature importance into non-relevant background areas, confirming that its architectural design enables it to make reliable detection decisions based only on the most necessary visual evidence.



Figure 39. Applying LIME XAI for feature/area contribution importance on nine images using YOLOv5nu.

4.4.2. YOLOv8n

In YOLOv8n, while the heatmaps successfully isolate the objects, showing fire in blue and smoke in red, as shown below on Figure 40, concentrated over the correct regions, the explanations are visually denser and slightly more diffused compared to the previous model. YOLOv8n frequently incorporates a wider perimeter of pixels around the fire and smoke plumes and utilizes a slightly more extensive feature set from the immediate background to confirm its highly accurate detections. This reliance on a broader, more context-rich feature map to achieve its high localization quality is visually confirmed by the LIME output.



Figure 40. Applying LIME XAI for feature/area contribution importance on nine images using YOLOv8n.

4.4.3. YOLOv10n

In YOLOv10n, XAI output presents a significant degree of fragmentation and instability in the model's feature selection when compared to the other cases. While the contributing features (fire in blue, smoke in red), as shown below on Figure 41, are broadly positioned over the correct objects, the heatmaps are frequently discontinuous and scattered. In several of the nine test images, the model's focus is clearly incomplete, failing to incorporate critical features like the base of a flame or the dense center of a smoke cloud. Furthermore, there is an increased tendency for fragmented importance to appear in irrelevant background regions, which is indicative of a less reliable and more confused decision-making process.



Figure 41. Applying LIME XAI for feature/area contribution importance on nine images using YOLOv10n.

4.4.4. YOLOv11n

In YOLOv11n, a near-optimal balance between focused interpretation and high accuracy is achieved. The heatmaps present an efficient feature selection strategy observed in YOLOv5n, yet the explanations retain the comprehensive object coverage associated with the higher-accuracy variants. Features contributing to 'fire' (blue) and 'smoke' (red), as shown below on Figure 42, are sharply localized and highly cohesive, maintaining robust object coverage without the excessive diffusion or background noise seen in YOLOv8n. In the same context, the explanations are highly stable and complete across all nine test images, visually confirming that the model's compact parameter count (2.6 million) does not compromise its ability to reliably and accurately extract necessary visual information.



Figure 42. Applying LIME XAI for feature/area contribution importance on nine images using YOLOv11n.

4.4.5. YOLOv12n

Lastly, the qualitative LIME analysis for YOLOv12n provides a clear visual correlation for its performance extremes—namely, its industry-leading localization accuracy achieved at the expense of speed. Across the nine test images, the heatmaps are visually the most detailed and extensive of all five variants. The features contributing to ‘fire’ (blue) and ‘smoke’ (red), as shown below on Figure 43, are highly concentrated, and the model has improved performance in incorporating the fringe pixels and boundary regions of the objects. The comprehensive, high-resolution coverage across the object boundaries visually explains the model’s significant computational load and its resultant penalty in inference speed, validating its supreme localization capability at the cost of real-time efficiency.



Figure 43. Applying LIME XAI for feature/area contribution importance to the nine images using YOLOv12n.

4.4.6. Integrating Quantitative Metrics with Qualitative XAI

While YOLOv12n achieved the highest overall localization accuracy mAP@0.50-0.95 of 0.5544, its LIME heatmaps revealed a highly detailed, boundary-intensive feature inspection process, which is directly associated with its significant performance bottleneck, causing the inference speed to drop below the critical 7 FPS real-time threshold (down to 6.2 FPS). Consequently, the quantitatively weakest model, YOLOv10n mAP@0.50 of 0.7906, was confirmed to be unreliable, showing fragmented and unstable feature selection across some test images.

Models YOLOv5n and YOLOv11n offered a better balance, with v11n demonstrating clean XAI explanations; however, their lower detection scores led to a narrower safety margin. Ultimately, YOLOv8n emerged as the optimal solution: it provided the highest detection accuracy mAP@0.50 of 0.8320 and, most critically, delivered the second-highest and robust inference speed (ranging from 7.5 to 9.5 FPS). The LIME analysis visually validated this overall balanced advantage, showing that YOLOv8n utilizes not the best but decent clean and focused feature extraction strategy.

4.5. YOLOv8n Model’s Failure Analysis Results

Below, Figures 44 and 45, provide a quantitative failure-case analysis of the YOLOv8n detector by jointly examining visually degraded inputs and their corresponding detection confidence scores. This systematic stress test exposes the model’s operational limits under adverse conditions, providing empirical justification for the integrated sensor fusion layer. As illustrated in Figure 44, the baseline image represents a high-contrast flame scenario, while successive perturbations introduce two levels of blurring, Gaussian noise, three levels of contrast degradation, partial occlusion, and small-scale fire events.



Figure 44. The images used to test the limitations of the chosen YOLO nano model (YOLOv8n).



Figure 45. The resulting images with detections and predictions from the chosen YOLO nano model (YOLOv8n).

The corresponding outputs in Figure 45 reveal the following performance trends:

- **Baseline and Scale Sensitivity:** In non-ideal conditions, where the flames were surrounded by other objects, YOLOv8n achieves stable boundary detections with confidence scores exceeding 0.50 in clearer views but up to 0.30 when not. However, in the small-fire scenario, detections are fragmented across multiple regions with confidence values ranging from 0.29 to 0.66, underscoring a lack of robustness to scale variation.
- **Blurring and Boundary Ambiguity:** Under increased blurring, the model continues to detect dominant flame structures, but detection boundaries become less precise, often merging individual flames into a single bounding box. Confidence values drop noticeably as blurring intensity increases (from 0.86/0.56 to 0.82/0.47), indicating a significant sensitivity to reduced feature sharpness.
- **Noise and Illumination Sensitivity:** Gaussian noise severely degrades certainty, with the model failing to identify all fire regions and scores dropping to a range of 0.26 to 0.31. In contrast-adjusted scenarios, moderate reduction yields confidence scores between 0.41 and 0.63, while high-intensity contrast actually improves confidence to 0.67–0.71. However, in low-light/darker backgrounds, the detector struggles, with fire and smoke predictions plateauing at 0.57–0.58.
- **Occlusions and Background Instability:** Occlusion by thick smoke presents a mixed failure mode, where partially obscured flames drop to 0.35 confidence, while isolated exposed regions reach 0.82. Notably, the smoke bounding regions occasionally include background clouds, reflecting an instability in distinguishing smoke from similar atmospheric features.

Collectively, these numeric trends reinforce that while YOLOv8n achieves strong overall performance, it exhibits clear confidence instability under noisy, low-contrast, and occluded conditions. These results delineate the reliability boundaries of the standalone vision modality and highlight the necessity of the multimodal fusion logic in maintaining system integrity in real-world industrial deployments.

4.6. Results of the Fused Decision-Only Sensor System

The final fusion system, illustrated below on Figure 46 as a flowchart as well, employs two LightGBM classifiers trained on the constructed fused dataset to predict the definitive FireAlert and SmokeAlert outputs. On the held-out test split, the classifiers achieved a performance of 1.000 across all metrics (Accuracy, F1-score, and ROC-AUC), as presented below on Figures 47–50. While an F1-score of 1.00 is typically indicative of overfitting in empirical detection tasks, in this framework, it serves as a formal verification of policy integrity.

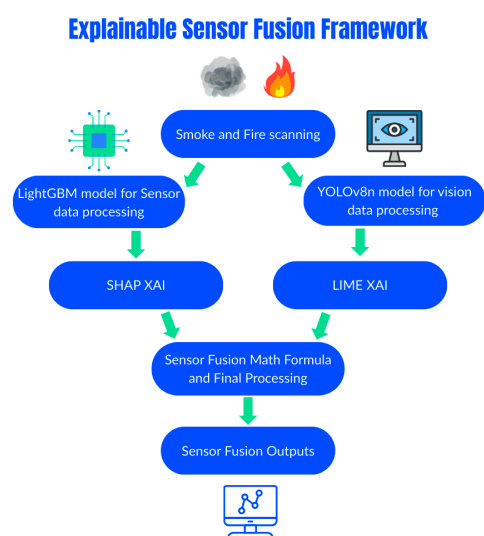


Figure 46. Visual flowchart of the architecture and functionality of the Explainable Sensor Fusion Framework.

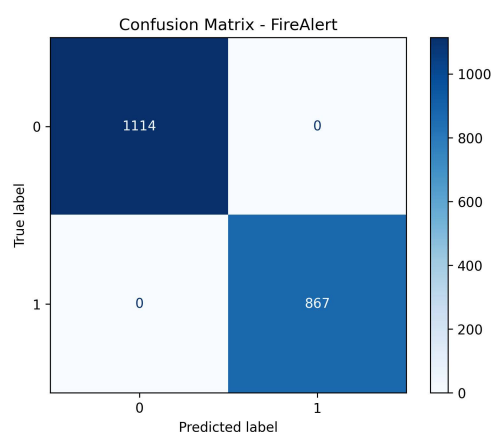


Figure 47. Confusion matrix of the sensor fusion system for fire detection.

These results demonstrate that the fusion layer has perfectly internalized the weighted-multiplicative logic and the conditional override rules defined in the methodology. This perfect convergence confirms that the decision-making process is deterministic and logically consistent, ensuring that there is no stochastic drift when the system tries to adjudicate between conflicting visual and sensor chemical readings as well as evidence. Thus, the

performance is not an empirical claim of real-world accuracy, which is established by the primary YOLO and sensor-ML models, but a mathematical proof that the logic engine operates with high fidelity to the safety requirements.

This design choice enables a focused evaluation of fusion behavior, interpretability, and internal decision consistency. The reported results primarily reflect the internal coherence and predictability of the fusion mechanism under controlled conditions, not real-world fire events.

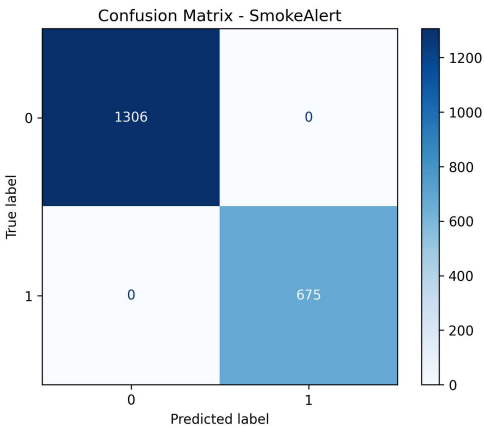


Figure 48. Confusion matrix of the sensor fusion system for smoke detection.

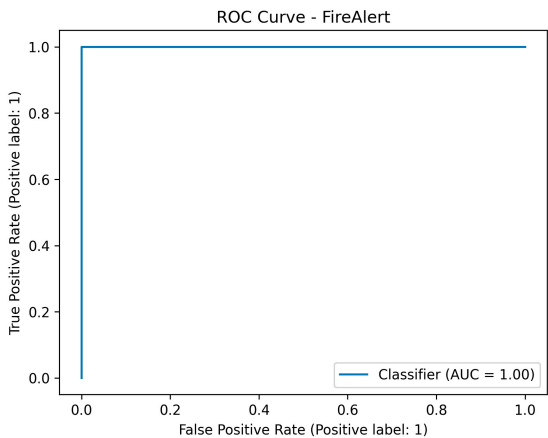


Figure 49. ROC metric performance results of the sensor fusion system for fire detection.

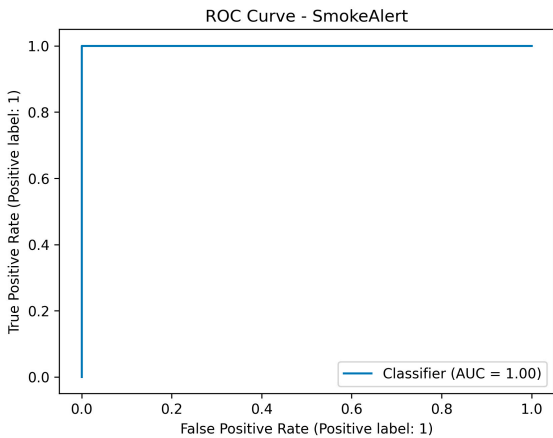


Figure 50. ROC metric performance results of the sensor fusion system for smoke detection.

4.7. Analysis of Fusion Behavior and State-Space Interpretability

The fused sensor dataset is constructed by augmenting the original IoT sensor dataset with probabilistic outputs generated by the independently trained models. Specifically, the sensor-based ML classifier provides P_{tabular} , while the vision model contributes P_{fire} and P_{smoke} . These probabilities are combined to compute $P_{\text{detection}}$ and to derive FireAlert and SmokeAlert labels based on predefined fusion rules and thresholds. By using a state-space alignment strategy rather than strict temporal synchronization, the dataset represents a comprehensive matrix of industrial combustion states, including “adversarial” scenarios (e.g., high visual confidence with baseline sensor readings) necessary to validate the framework’s logic. The resulting dataset is split into training and testing subsets using stratified sampling to preserve alert distributions. This synthesized dataset is intended for the controlled evaluation of fusion behavior rather than real-world deployment.

4.8. XAI Analysis of the Fused Sensor System

In this step, interpretability analysis was conducted using SHAP. The results presented below on Figures 51 and 52, confirm that the final LightGBM decision-makers place the highest reliance on the derived model probabilities rather than raw sensor inputs alone, validating the strength of the fusion design. For the Fire Alert model, the decision was dominated by P_{fire} (YOLO’s fire probability), which exhibited an SHAP importance of 0.30, followed by the combined vision probability $P_{\text{detection}}$ (0.24) and the tabular sensor probability P_{tabular} (0.14). This hierarchy confirms that visual confirmation is paramount for initiating a fire alarm.

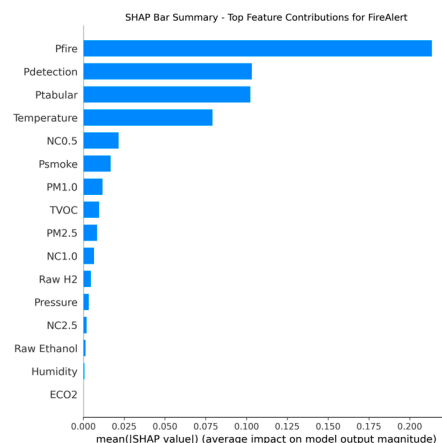


Figure 51. Sensor fusion features that contributed the most to the fire detection decision-making process.

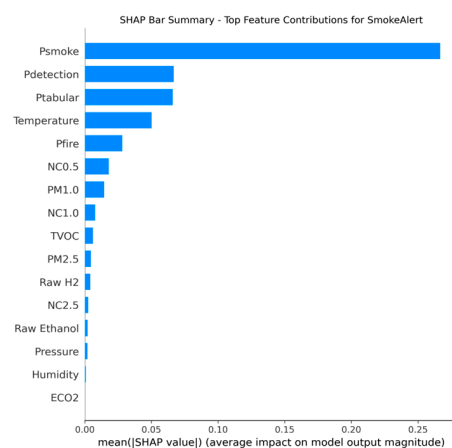


Figure 52. Sensor fusion features that contributed the most to the smoke detection decision-making process.

Conversely, the Smoke Alert model was similarly dominated by P_{smoke} (YOLO's smoke probability) (0.49), but the relative importance of $P_{tabular}$ (0.09) closely trailed the general vision probability $P_{detection}$, highlighting the greater necessity of the sensor component in validating the often diffuse- and context-sensitive nature of smoke detection. Temperature was the most influential raw sensor feature in both models, providing essential contextual confirmation for thermal events.

5. Discussion

Taking everything into account, we can conclude that the Explainable Sensor Fusion Framework provided better results than both the sensor-only and vision-only approaches, in an overall context, because its advantage is that it uses strengths from both worlds, while at the same time addressing their main weaknesses, which thus resulted in a model that provides high robustness and reliability for smoke and fire detection purposes.

More specifically, we noticed that the sensor-only system has high numerical accuracy in a controlled environment, but it is quite limited by its lack of spatial generalization, which in turn does not cover all the smoke/fire cases. It is mostly dependent on unstable features such as TVOC and Pressure, which makes it unable to confirm the location or presence of a fire event outside its physical range. In contrast, the vision-only system (YOLOv8n which achieved the best mAP@0.50 results) offers excellent spatial coverage and accurate localization of smoke and fire cases, as validated by the application of the LIME XAI method. However, it is still vulnerable to false positives from environmental factors, such as reflections, colored objects, and tiny flames, that could be meaningless or used by machines in factories or for other purposes. Additionally, it operates at a speed that, although considered quick, sometimes may risk delays in critical real-time situations (7.5 to 9.5 FPS).

Moreover, our designed Decision-Level Fusion System is able to address the trade-offs caused by these two individual approaches. By using a “late fusion” strategy, we can independently process sensor and visual data and then combine their probabilities through a weighted-multiplicative rule. The perfect scores achieved on our synthesized fusion dataset demonstrate the internal consistency and logical reliability of this decision-only fusion mechanism, and prove that the system follows the intended logic of cross-verifying a visual fire with environmental evidence.

The conducted SHAP XAI analysis supports this design as well, showing that the final alarm decision depends mainly on the high precision of the visual component (P_{fire} or P_{smoke}) for primary detection and then considers the sensor's readings for confirmation. On the other hand, the system strengthens this signal using the $P_{tabular}$ and temperature features to obtain a more robust and essential context. This modular approach not only ensures a transparent decision-making process but also establishes a verifiable framework for multimodal safety systems.

Finally, to elaborate even more on the robustness of the proposed framework, we conducted a validation to analyze specific “corner cases” where the individual modalities reached their algorithmic limits. In “Visual Deception” scenarios, such as high-intensity reflections from metallic surfaces or dynamic industrial lighting, the YOLOv8n model occasionally generated false positives with confidence scores exceeding 0.70. However, because these optical artifacts were not accompanied by deviations in ambient chemistry (sensor readings), the P_{sensor} remained at baseline, and the weighted-multiplicative fusion rule effectively suppressed the alert by failing to reach the 0.65 confirmation threshold.

Conversely, the system addressed “Sensor Transport Lag” in tests where the fire source (ignited papers that caused smoke and flames) was distant from the sensor node. In that case, airflow dilution and physical distance caused a significant delay in chemical detection;

however, the High-Confidence Visual Override ($P_{yolo} \geq 0.85$) proved critical, leveraging zero-latency line-of-sight detection to trigger an immediate alert and bypass the sensor-induced latency. These cases demonstrate that while individual modalities are subject to specific failures, for instance, reflections for vision and transport lag for sensors, the integrated architecture ensures operational continuity by leveraging the spatial-temporal decoupling of the two sensing streams.

6. Conclusions

In this work, an interpretable and resource-aware hybrid sensor-vision fusion framework for smoke and fire detection was designed and evaluated. The initial performance analysis of multiple YOLO nano architectures highlighted a critical trade-off in modern object detection: although newer models such as YOLOv12n offered marginal improvements in localization accuracy, their inability to consistently satisfy real-time constraints (≥ 7 FPS) rendered them unsuitable for immediate edge deployment. The qualitative explainability analysis using LIME provided essential insight into the visual reasoning of each detector, supporting the selection of YOLOv8n as the most balanced solution. Its favorable mAP@50 performance (0.8320), stable inference speed (7.5–9.5 FPS), and clean, focused explanations collectively confirmed its architectural efficiency and practical reliability.

Building upon this vision backbone, the proposed system integrates environmental sensor information through a weighted decision-level fusion strategy, combining probabilistic outputs from both sensing modalities. On the constructed fusion dataset, the integrated framework achieved perfect classification scores across all reported metrics. These results indicate that the fusion logic operates consistently and predictably within the defined experimental setup and effectively mitigates individual failure modes of vision-only systems (e.g., visual clutter and reflections) and sensor-only systems (e.g., limited spatial coverage). However, these perfect fusion metrics should be interpreted as a validation of the internal coherence and interpretability of the decision-level fusion mechanism under controlled conditions, rather than as definitive evidence of real-world generalization or deployment readiness. Further evaluation on synchronized, event-level sensor-image datasets remains a necessary step toward operational validation in industrial environments. Finally, future work includes, among others:

1. A formal sensitivity analysis to systematically evaluate how variations in the fusion weights (e.g., sensor vs. YOLO importance) and decision thresholds (confirmation and override levels) influence the detection performance, robustness, and false-alarm rates of the ESF system.
2. Investigating more sophisticated fusion mechanisms beyond the weighted product approach, such as Attention-based Neural Networks trained end-to-end on the combined sensor and image features, could potentially yield even higher reliability and resilience to noise.
3. The current system's perfect scores should be validated against a larger, more diverse dataset captured under varying lighting, weather, and obscuration conditions, especially to ensure the model maintains performance against false alarm scenarios (e.g., steam, brightly colored objects, sunsets).
4. The current system is being used in a controlled environment to showcase the robustness of the fusion approach, compared to sensor-only or vision-only choices. The next step for our implementation will be to use real-time sensor readings from actual sensors and smoke/fire detections, to demonstrate the effectiveness of the fusion approach in real-world scenarios.

5. The YOLO nano models were trained on 20 epochs to test their limits and capabilities. Enriching the models with more epochs and training would lead to even better and promising results.
6. The current experiments are conducted on an x86 laptop (Dell, Psachna, Greece). For more insights regarding edge device performance, we will benchmark the selected lightweight models on embedded edge hardware, such as the Raspberry Pi 5 mini computer (Raspberry PI, Psachna, Greece), to validate real-time performance and resource efficiency under deployment conditions.
7. For real-world deployment, each visual frame should be temporally aligned with sensor readings to ensure accurate fusion. This synchronization would improve robustness, reduce spurious correlations, and enable reliable detection in dynamic environments. Thus, future work will focus as well as validate the fusion system with synchronized multimodal data streams.

Author Contributions: Conceptualization, E.D. and P.K.G.; methodology, E.D. and P.K.G.; software, E.D. and P.K.G.; validation, E.D. and P.K.G.; formal analysis, E.D. and P.K.G.; investigation, E.D. and P.K.G.; resources, E.D. and P.K.G.; data curation, E.D. and P.K.G.; writing—original draft preparation, E.D. and P.K.G.; writing—review and editing, E.D. and P.K.G.; visualization, E.D. and P.K.G.; supervision, P.K.G.; project administration, E.D. and P.K.G.; funding acquisition, P.K.G. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The data presented in this study are available on request from the corresponding author due to privacy and ethical restrictions.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

AI	Artificial Intelligence
API	Application Programming Interface
DL	Deep Learning
NN	Neural Network
CNN	Convolutional Neural Network
EFB	Exclusive Feature Bundling
ESF	Explainable Sensor Fusion
FNN	Feedforward Neural Network
FPS	Frames per Second
HPO	Hyperparameter Optimization
IoU	Intersection over Union
GOSS	Gradient-based One-Side Sampling
GBDT	Gradient Boosting Decision Trees
KNC	K-Neighbors Classifier
LightGBM	Light Gradient Boosting Machine
LIME	Local Interpretable Model-agnostic Explanations
mAP	Mean Average Precision
ML	Machine Learning
MLP	Multilayer Perceptron
RNN	Recursive Neural Networks
SHAP	SHapley Additive exPLANATIONS
SVC	Support Vector Classifier
TPE	Tree-structured Parzen Estimator

TVOC	Total Volatile Organic Compounds
XAI	Explainable Artificial Intelligence
XGBoost	eXtreme Gradient Boosting
YOLO	You Only Look Once

References

1. Peruzzi, G.; Pozzebon, A.; Van Der Meer, M. Fight Fire with Fire: Detecting Forest Fires with Embedded Machine Learning Models Dealing with Audio and Images on Low Power IoT Devices. *Sensors* **2023**, *23*, 783. [\[CrossRef\]](#)
2. Ali, M.L.; Zhang, Z. The YOLO framework: A comprehensive review of evolution, applications, and benchmarks in object detection. *Computers* **2024**, *13*, 336. [\[CrossRef\]](#)
3. Magdin, M.; Balogh, Z. Comparison classification algorithms and the YOLO method for video analysis and object detection. *Sci. Rep.* **2025**, *15*, 25432. [\[CrossRef\]](#) [\[PubMed\]](#)
4. Hasan, R.H.; Hassoo, R.M.; Aboud, I.S. YOLO versions architecture: Review. *Int. J. Adv. Sci. Res. Eng. (IJASRE)* **2023**, *9*, 73–92. [\[CrossRef\]](#)
5. Kang, S.; Hu, Z.; Liu, L.; Zhang, K.; Cao, Z. Object detection YOLO algorithms and their industrial applications: Overview and comparative analysis. *Electronics* **2025**, *14*, 1104. [\[CrossRef\]](#)
6. Mela, J.L.; García Sánchez, C. YOLO-based power-efficient object detection on edge devices for USVs. *J. Real-Time Image Process.* **2025**, *22*, 108. [\[CrossRef\]](#)
7. Polenakis, I.; Sarantidis, C.; Karydis, I.; Avlonitis, M. A comparative study of YOLO algorithm variants. *Signals* **2025**, *6*, 60. [\[CrossRef\]](#)
8. Zhou, Y.; Guo, Z.; Dong, Z.; Yang, K. TensorRT implementations of model quantization on edge SoC. In Proceedings of the 2023 IEEE 16th International Symposium on Embedded Multicore/Many-Core Systems-on-Chip (MCSoc), Singapore, 18–21 December 2023; pp. 486–493. [\[CrossRef\]](#)
9. Dong, Q.; Han, T.; Wu, G.; Sun, L.; Huang, M.; Zhang, F. Industrial device-aided data collection for real-time rail defect detection via a lightweight network. *Eng. Appl. Artif. Intell.* **2025**, *141*, 112102. [\[CrossRef\]](#)
10. Feng, H.; Mu, G.; Zhong, S.; Zhang, P.; Yuan, T. Benchmark analysis of YOLO performance on edge intelligence devices. *Cryptography* **2022**, *6*, 16. [\[CrossRef\]](#)
11. Han, B.-G.; Lee, J.-G.; Lim, K.-T.; Choi, D.-H. Design of a scalable and fast YOLO for edge-computing devices. *Sensors* **2020**, *20*, 6779. [\[CrossRef\]](#)
12. Zhang, Z.; Yang, Y.; Xu, X.; Liu, L.; Yue, J.; Ding, R.; Lu, Y.; Liu, J.; Qiao, H. GVC-YOLO: A Lightweight Real-Time Detection Method for Cotton Aphid-Damaged Leaves Based on Edge Computing. *Remote Sens.* **2024**, *16*, 3046. [\[CrossRef\]](#)
13. Li, J.; Ye, J. Edge-YOLO: Lightweight infrared object detection method deployed on edge devices. *Appl. Sci.* **2023**, *13*, 4402. [\[CrossRef\]](#)
14. Aryal, Y. Assessing Climate and Watershed Controls on Rain-on-Snow Runoff Using XGBoost-SHAP Explainable AI (XAI). *Geosciences* **2025**, *15*, 467. [\[CrossRef\]](#)
15. Tahir, H.A.; Alayed, W.; Hassan, W.U.; Haider, A. A Novel Hybrid XAI Solution for Autonomous Vehicles: Real-Time Interpretability Through LIME-SHAP Integration. *Sensors* **2024**, *24*, 6776. [\[CrossRef\]](#)
16. del Castillo Torres, G.; Roig-Maimó, M.F.; Mascaró-Oliver, M.; Amengual-Alcover, E.; Mas-Sansó, R. Understanding How CNNs Recognize Facial Expressions: A Case Study with LIME and CEM. *Sensors* **2023**, *23*, 131. [\[CrossRef\]](#)
17. Pang, B.; Nijkamp, E.; Wu, Y.N. Deep Learning With TensorFlow: A Review. *J. Educ. Behav. Stat.* **2020**, *45*, 227–248. [\[CrossRef\]](#)
18. Chicho, B.T.; Bibo Sallow, A. A comprehensive survey of deep learning models based on Keras framework. *J. Soft Comput. Data Min.* **2021**, *2*, 49–62. [\[CrossRef\]](#)
19. Pedregosa, F.; Varoquaux, G.; Gramfort, A.; Michel, V.; Thirion, B.; Grisel, O.; Blondel, M.; Prettenhofer, P.; Weiss, R.; Dubourg, V.; et al. Scikit-learn: Machine Learning in Python. *J. Mach. Learn. Res.* **2011**, *12*, 2825–2830. Available online: <https://www.jmlr.org/papers/v12/pedregosa11a.html> (accessed on 12 January 2026).
20. Stoltzfus, J.C. Logistic regression: A brief primer. *Acad. Emerg. Med.* **2011**, *18*, 1099–1104. [\[CrossRef\]](#) [\[PubMed\]](#)
21. Dhananjay, B.; Sivaraman, J. Analysis and classification of heart rate using CatBoost feature ranking model. *Biomed. Signal Process. Control.* **2021**, *68*, 102610. [\[CrossRef\]](#)
22. Suyal, M.; Goyal, P. A review on analysis of K-Nearest Neighbor classification machine learning algorithms based on supervised learning. *Int. J. Eng. Trends Technol.* **2022**, *70*, 43–48. [\[CrossRef\]](#)
23. Valkenborg, D.; Rousseau, A.J.; Geubbelmans, M.; Burzykowski, T. Support vector machines. *Am. J. Orthod. Dentofac. Orthop.* **2023**, *164*, 754–757. [\[CrossRef\]](#)
24. Priyam, A.; Abhijeet; Gupta, R.K.; Srivastava, S. Comparative analysis of decision tree classification algorithms. *Int. J. Curr. Eng. Technol.* **2013**, *3*, 334–337. Available online: <https://inpressco.com/wp-content/uploads/2013/03/Paper17334-3371.pdf> (accessed on 4 February 2026).

25. Zhang, L.; Liu, K.; Wang, Y.; Omariba, Z.B. Ice Detection Model of Wind Turbine Blades Based on Random Forest Classifier. *Energies* **2018**, *11*, 2548. [CrossRef]
26. Torlay, L.; Perrone-Bertolotti, M.; Thomas, E.; Baciú, M. Machine learning–XGBoost analysis of language networks to classify patients with epilepsy. *Brain Inform.* **2017**, *4*, 159. [CrossRef]
27. Gan, M.; Pan, S.; Chen, Y.; Cheng, C.; Pan, H.; Zhu, X. Application of the machine learning LightGBM model to the prediction of the water levels of the lower Columbia River. *J. Mar. Sci. Eng.* **2021**, *9*, 496. [CrossRef]
28. Chang, W.; Wang, X.; Yang, J.; Qin, T. An Improved CatBoost-Based Classification Model for Ecological Suitability of Blueberries. *Sensors* **2023**, *23*, 1811. [CrossRef] [PubMed]
29. Cao, Y. An MLP classifier for prediction of HBV-induced liver cirrhosis using routinely available clinical parameters. *Dis. Markers* **2013**, *35*, 653–660. [CrossRef] [PubMed]
30. Mishra, C.; Gupta, D.L. Deep machine learning and neural networks: An overview. *IAES Int. J. Artif. Intell. (IJ-AI)* **2017**, *6*, 66–73. [CrossRef]
31. Singh, M.; Akula, A. A comparative study of YOLO-V5 variants performance for object detection in thermal infrared images. In Proceedings of the 2024 IEEE 5th India Council International Subsections Conference (INDISCON), Chandigarh, India, 22–24 August 2024; pp. 1–6. [CrossRef]
32. Wang, J.; Qi, Z.; Wang, Y.; Liu, Y. A lightweight weed detection model for cotton fields based on an improved YOLOv8n. *Sci. Rep.* **2025**, *15*, 457. [CrossRef]
33. Alkhamash, E.H. A Comparative Analysis of YOLOv9, YOLOv10, YOLOv11 for Smoke and Fire Detection. *Fire* **2025**, *8*, 26. [CrossRef]
34. Wang, Q.; Wang, X.; Hou, J.; Liu, X.; Wen, H.; Ji, Z. MF-YOLOv10: Research on the Improved YOLOv10 Intelligent Identification Algorithm for Goods. *Sensors* **2025**, *25*, 2975. [CrossRef]
35. Gao, L.; Cao, H.; Zou, H.; Wu, H. DMN-YOLO: A robust YOLOv11 model for detecting apple leaf diseases in complex field conditions. *Agriculture* **2025**, *15*, 1138. [CrossRef]
36. He, L.; Zhou, Y.; Liu, L.; Cao, W.; Ma, J. Research on object detection and recognition in remote sensing images based on YOLOv11. *Sci. Rep.* **2025**, *15*, 14032. [CrossRef]
37. Ge, Y.-T.; Wang, B.-J.; Sun, S.; Lan, Y.-B. Lightweight Power-Line Visual Detection in Agricultural UAV Scenarios Based on an Improved YOLOv12n Model. *Sensors* **2025**, *26*, 109. [CrossRef]
38. Bakir, C.; Gezer, A. Real-Time Automatic Detection of Nutrient Deficiency in Lettuce Plants With New YOLOV12 Model. *J. Sens.* **2025**, *2025*, 5592225. [CrossRef]
39. Graham, J.; Sheng, V.S. Scalable SHAP-Informed Neural Network. *Mathematics* **2025**, *13*, 2152. [CrossRef]
40. Rezk, N.G.; Alshathri, S.; Sayed, A.; El-Din Hemdan, E.; El-Beheri, H. XAI-Augmented Voting Ensemble Models for Heart Disease Prediction: A SHAP and LIME-Based Approach. *Bioengineering* **2024**, *11*, 1016. [CrossRef] [PubMed]
41. Duță, Ș.; Sultana, A.E. Optimizing Depression Classification Using Combined Datasets and Hyperparameter Tuning with Optuna. *Sensors* **2025**, *25*, 2083. [CrossRef] [PubMed]
42. Dibra, E. Explainable Smoke and Fire Detection System [Code Repository]. *GitHub*. 2025. Available online: https://github.com/EndriDibra/Explainable_Smoke_And_Fire_Detection_System/tree/main (accessed on 28 December 2025).
43. Blattmann, S. Smoke Detection Dataset [Dataset]. *Kaggle*. 2023. Available online: <https://www.kaggle.com/datasets/deepcontractor/smoke-detection-dataset> (accessed on 27 December 2025).
44. Menon, G. Fire and Smoke (Roboflow) [Dataset]. *Kaggle*. 2023. Available online: <https://www.kaggle.com/datasets/gautamrmenon/fire-and-smoke-roboflow> (accessed on 27 December 2025).
45. Mark, J. Where There's Fire, There's Smoke. *The New York Times, Opinion*, 8 September 2017. Available online: <https://www.nytimes.com/2017/09/08/opinion/where-theres-fire-theres-smoke.html> (accessed on 27 December 2025).
46. Egan, C. Wildfire Smoke and Your Health. *Steinbach First Aid*, 30 July 2025. Available online: <https://steinbachfirstaid.com/wildfire-smoke-and-your-health/> (accessed on 27 December 2025).
47. Wikipedia. The Free Encyclopedia. Smoke. Available online: <https://en.wikipedia.org/wiki/Smoke> (accessed on 27 December 2025).
48. iStockphoto. Fire in the City Overview. Getty Images. Available online: https://media.istockphoto.com/id/494853349/photo/fire-in-the-city-overview.jpg?s=612x612&w=0&k=20&c=J9w2-ys_Y79cbIBJjgw9Ybtw2QrL4ZiSLf39jE77pW0= (accessed on 27 December 2025).
49. Dunstan, J.; Rizmal, Z. Melbourne Factory Fire at Campbellfield's Bradbury Plant Sends Smoke over Suburbs. *ABC News*. 2019. Available online: <https://www.abc.net.au/news/2019-04-05/campbellfield-factory-fire-sends-smoke-over-melbournes-north/10973650> (accessed on 27 December 2025).
50. Vecteezy. A Large Industrial Plant with a Large Fire in the Background. Available online: <https://www.vecteezy.com/video/50735718-a-large-industrial-plant-with-a-large-fire-in-the-background> (accessed on 27 December 2025).

51. Vecteezy. A Large Industrial Plant with Lots of Smoke Coming out of It. Available online: <https://www.vecteezy.com/video/50735570-a-large-industrial-plant-with-lots-of-smoke-coming-out-of-it> (accessed on 27 December 2025).
52. IMEC Technologies. Causes of Fires in Manufacturing Plants. 2020. Available online: <https://www.imectechologies.com/2020/12/15/causes-of-fires-in-manufacturing-plants/> (accessed on 27 December 2025).
53. Millennium Fire Protection. Most Common Causes of Factory Fires. Available online: <https://www.m-f-p.co.uk/news/most-common-causes-of-factory-fires/> (accessed on 27 December 2025).

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.